过滤器透水数据分析及其最优维护方案

摘要

本文围绕工厂污水处理中过滤器的透水率退化与维护优化问题，提出了从数据规律提取到维护决策建模的完整分析框架。首先，对 10 台过滤器两年间的透水率监测数据进行预处理与趋势分析，识别出周期性波动与长期衰减特征，并通过分层效应剥离法分离季节因素与维护类型的耦合影响，量化各过滤器的基准维护能力与季节修正因子。其次，基于分设备独立衰减速率与固定维护规则，构建以月为步长的寿命预测模型，实现剩余寿命的稳健估计。进而，以年均总成本最小化为目标，建立考虑维护与更换的双阈值优化模型，通过全域枚举确定最优维护触发策略。最后，通过成本波动与参数扰动下的灵敏度分析，验证了优化方案的鲁棒性。本文的方法框架可为工业过滤设备的低成本、高可靠运维管理提供量化决策支持。

关键词：过滤器；透水率衰减；寿命预测；维护优化；成本最小化；季节因子；灵敏度分析

目录

1 问题重述 **4**
1.1 问题背景 4
1.2 研究任务 4

2 数据分析 **5**
2.1 数据的周期性、下降趋势和规律 5
2.2 透水率变化特征及影响量化指标分析 7

3 透水率变化的影响指标 **8**
3.1 建模思路与整体框架 8
3.2 季节影响、自然衰减、维护效果的量化处理 9
3.2.1 两步平均法季节因子计算 9
3.3 上述指标的量化结果 11

4 过滤器寿命预测 **14**
4.1 寿命预测模型 14
4.2 10 台过滤器的剩余寿命预测结果 15

5 基于年均成本最小的最优维护方案模型 **16**
5.1 思路说明 17
5.2 约束条件设计 18
5.2.1 状态物理约束 18
5.2.2 系统运行约束 18
5.2.3 成本经济约束 18
5.3 模型求解与最优方案 19

6 成本波动下维护方案灵敏度分析 **20**
6.1 价格波动场景设置 20
6.2 方案可行性分析 20
6.3 模型鲁棒性检验 21

7 模型评价与推广 **22**
7.1 模型优点 22
7.1.1 简化方案严格契合题目核心要求 22
7.1.2 复杂度控制适配工程实操场景 22
7.1.3 核心指标量化满足题目分析需求 22
7.1.4 成本优化逻辑贴合工业运维目标 22
7.2 模型不足 23

7.2.1 核心参数计算滥用平均算法，精度严重不足 23
7.2.2 季节效应量化采用极端理想化假设 23
7.2.3 透水率演化采用月度近似迭代，累积误差较大 23
7.2.4 维护增益评估方式片面，抗干扰能力极差 23
7.2.5 小样本数据导致参数鲁棒性低下 23
7.2.6 环境-性能耦合机制刻画极度浅显 23
7.2.7 维护决策采用静态固定阈值，无自适应能力 23
7.3 模型改进与推广价值 24
7.3.1 模型系统性改进方向 24
7.3.2 模型推广应用价值 24

8 参考文献 **25**

9 附录 **26**
9.1 程序运行环境 26
9.2 完整源代码 26
9.3 支撑材料说明 65

1 问题重述

1.1 问题背景

工业污水排放必须满足严格的净化标准, 过滤处理是保障污水达标排放的核心环节。某工厂采用多台污水过滤器并联运行的模式开展污水处理工作，透水率作为衡量过滤器性能与运行状态的关键指标，其变化直接影响污水净化效果与设备运行成本。

随着过滤作业持续进行，污染物附着堆积、设备自然老化等因素会导致透水率呈长期下降趋势。当设备年平均透水率低于阈值（37）且维护无法有效恢复性能时，过滤器将达到寿命终点，需进行更换。为延缓透水率衰减、延长设备使用寿命，工厂会对过滤器开展分级维护作业，包括成本可忽略的日常小维护、中维护和大维护三类，不同维护方式对透水率的提升效果、持续时长及对设备的损耗程度存在显著差异，且维护效果受季节因素影响，表现出明显的波动特征。

设备购置、维护与更换均会产生成本支出，如何在保障污水净化达标、设备稳定运行的前提下，通过科学的维护与更换决策，降低工厂污水净化的年均成本，成为工厂运维管理面临的关键问题。

1.2 研究任务

结合题目给出的 10 台过滤器 2024 年 4 月至 2026 年 4 月的透水率监测数据、维护记录及成本参数，本研究的核心任务分为以下四项：数据处理与规律分析：对原始透水率数据进行处理，消除随机波动影响，分析透水率的周期性变化特征与长期下降趋势；量化不同维护方式对透水率的提升效果与衰减规律，结合季节因素分析维护效果的差异，提炼影响透水率变化的关键指标。设备寿命预测：在第 1 问分析结果的基础上，综合考虑透水率自然衰减、季节影响、维护效果等因素，构建过滤器寿命预测模型；基于当前固定维护规律，对 10 台过滤器的剩余使用寿命进行预测，明确设备更换节点。最优维护方案优化：以工厂污水净化年均成本最小化为目标，综合考虑设备购置成本、维护成本、寿命周期约束、透水率达标约束，建立多约束优化模型，制定兼顾成本与性能的最优维护与更换方案。方案灵敏度分析：针对过滤器购置价格、维护成本的波动场景，分析最优维护方案的可行性变化，检验方案的鲁棒性，为工厂应对成本波动提供决策参考。

2 数据分析

2.1 数据的周期性、下降趋势和规律

为了比较清晰地看到不同过滤器的过滤变化情况，我们利用 Python 对 Excel 格式的原始监测数据进行**数据提取与处理**，绘制得到 10 台过滤器在 2024 年 4 月—2026 年 4 月观测周期内的**日平均透水率变化趋势图**与**月透水率变化折线图**，其中日变化趋势图的目的是为了展现**维护过程（即断点）**，如下面两图所示。

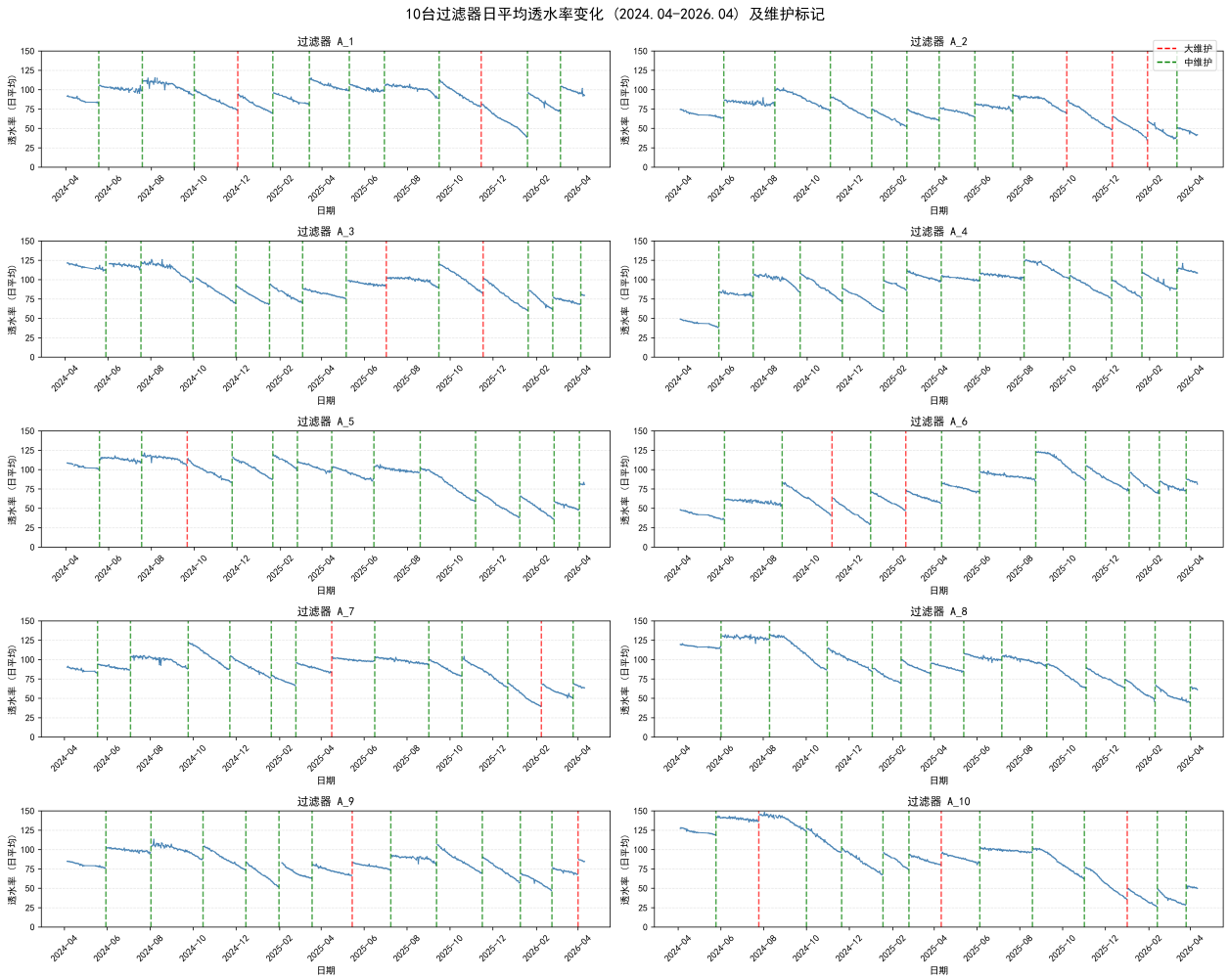


图 1: 过滤器平均透水率变化（2024.04–2026.04）

绿虚线为中维护，红虚线为大维护。**维护效果呈现显著季节性差异：冬季是两类维护的黄金窗口期，透水率提升率均为四季最高，且大维护的季节差异远大于中维护。中维护四季效果相对均衡，春季次之，夏秋略低；大维护仅冬季效果突出，春夏两季提升率极低，几乎无明显收益。**不同过滤器的具体维护表现存在差异。

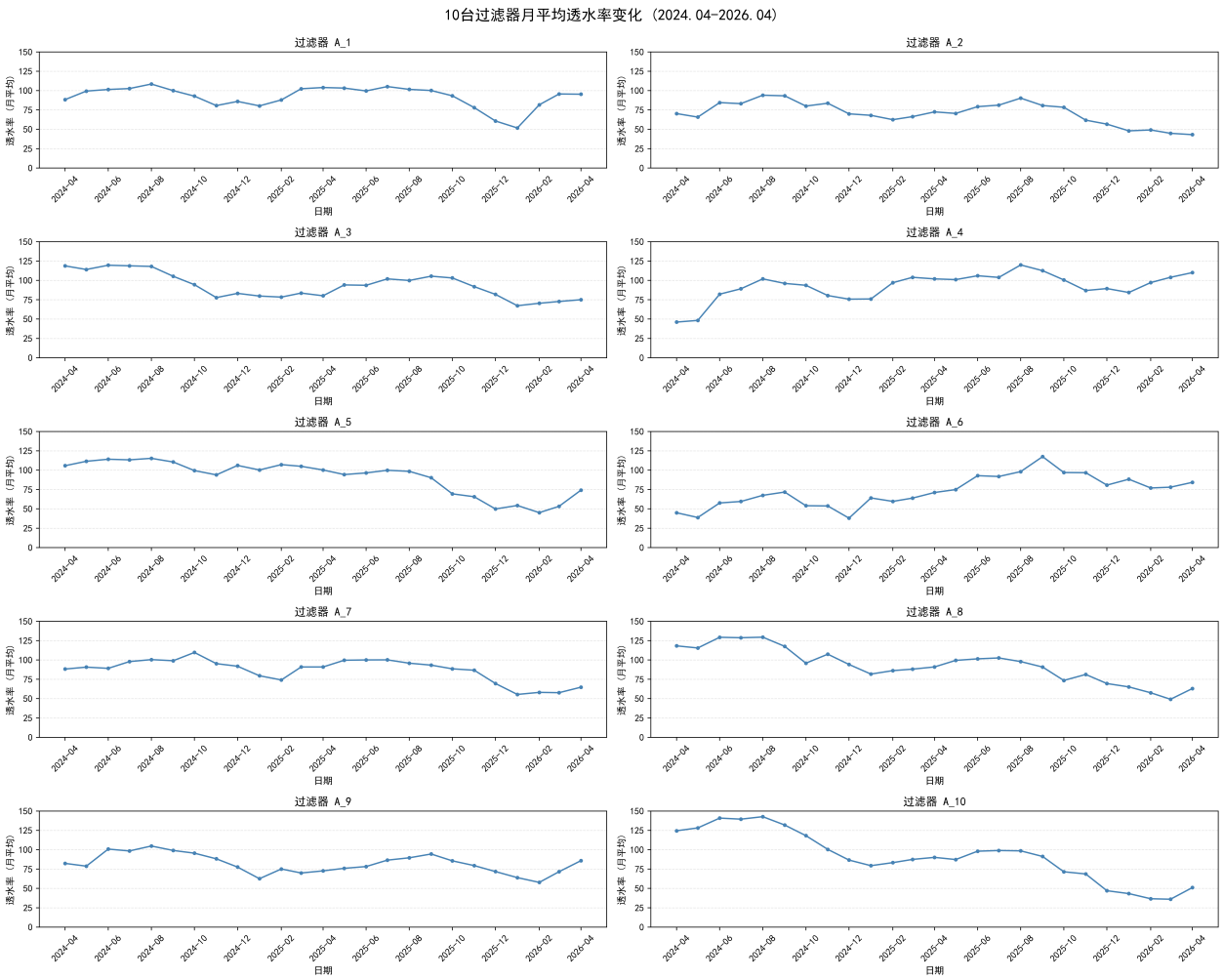


图 2: 月透水率变化折线图（2024.04–2026.04）

综合考虑趋势图，整体而言，十台过滤器**透水率变化均呈现“设备老化”与“维护修复”交替作用的波动特征**，不存在单纯单调变化趋势，不同设备运行表现差异显著。2024 年 4 月至 2026 年 4 月全周期内，**设备运行大致分为前期高位稳定、中期波动衰减、后期加速老化三阶段**，结合图一断点末期大型维护仅对部分设备起到明显修复作用。

逐台对比，各设备**运行轨迹分化明显**：A4、A6 整体运行稳定，**老化缓慢，性能稳中有升**；A1、A3、A5、A7、A8、A10 呈**“前期高位、后期加速衰减”特征**，部分设备末期维护效果显著，部分则收效甚微；A2 整体趋势持续向下，常规维护几乎无法扭转衰减；A9 **透水率反复波动，维护频次虽高，但效果持续时间短，运行状态不稳定**。

结合数据特征与运行背景，我们初步推断**影响透水率演变的主要因素包括四类**：**设备初始性能与运行负荷差异**；**维护干预时机与类型，早期维护效果明显优于后期**；**季节变化带来的污染负荷波动，间接影响设备老化速率**；**长期运行中污染物持续累积，是导致性能不可逆下降的主要外部原因**，此处我们统一归类为**自然衰减**。

2.2 透水率变化特征及影响量化指标分析

为**剥离季节与维护类型对透水率的独立影响**，本研究通过以下方法**消除设备自然衰减的干扰**：首先抛出维护前后几日的突变值后，**计算相邻两次维护之间透水率的日平均下降率，剔除偶然数据与异常值**后，基于两年运行数据**进行二次平均**，得到**设备自然衰减的基准水平**。在此基础上，**计算不同维护方式在各季节下的透水率相对提升率**，以此**表征维护措施的实际恢复效果**。为简化分析，假设同一维护方式在相同季节下的**作用效果具有一致性**，即**维护效果仅受季节与维护类型的交互影响**。为直观呈现维护效果的分布特征，基于 10 台过滤设备的历史数据，通过 Python 绘制箱型图（图 3），并经过多设备交叉验证与重复检验，结果稳定可靠。

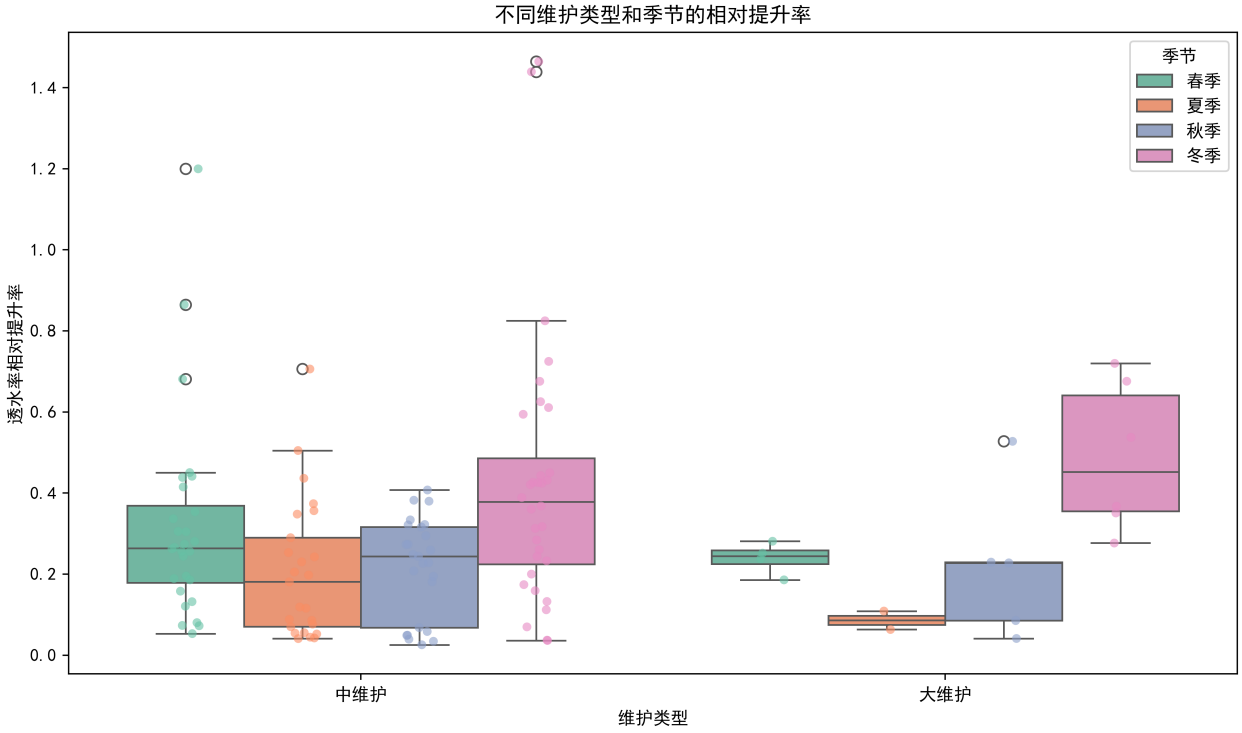


图 3: 维护类型与季节对透水率作用的箱型图

从维护类型看，**中维护整体恢复效果显著优于大维护**：**各季节透水率提升率中位数均更高，数据离散度更大，工况依赖性更强**；**大维护提升率整体偏低**，夏、秋季中位数不足 0.2，效果有限。从季节看，**冬季维护效果在两类维护中均最突出**：**中、大维护冬季提升率中位数约 0.4 与 0.5，显著高于其他季节，数据集中于较高水平**；**春季次之，夏、秋季偏低**，其中夏季大维护中位数不足 0.1，为图中最低值。箱型图离群点反映了**极端工况差异**：**中维护春、冬季存在提升率超 1.0 的高值，特定条件下可大幅恢复透水率**；**大维护离群点少、分布集中，效果稳定但上限不足**。上述结果为后续**维护策略的成本-效益优化**提供了**量化依据**。

3 透水率变化的影响指标

3.1 建模思路与整体框架

结合题意，小维护修复效果微弱且维护成本极低，不纳入定量分析，本文仅针对中维护与大维护展开研究。为分离**季节独立影响**、消除**维护类型差异**与**样本不均衡**导致的模型偏差，采用分层效应剥离法构建季节修正因子，实现透水率衰减与维护增益的动态修正。

季节通过水温、进水浊度、排污组分等环境变量，对透水率衰减与维护修复效果产生耦合驱动作用：恶劣工况季节透水率衰减速率快，维护修复增益显著；温和工况季节衰减平缓，维护提升效果较弱。依据现实情况考虑，**季节因素对每一台滤水器的作用应当是几乎一致的**；而**大、中维护效果对每台滤水器具有不同作用效果**，因此需要分别考虑不同滤水器经过大、中维护的实际效果。二者作用方向具有一致性，因此采用统一季节修正系数具备物理合理性，可保障模型的可辨识性与求解稳定性。

本文以单次维护记录为基础统计单元，区分**过滤器个体**、**维护类型**与**季节**三类维度，通过交叉分组统计、加权校正、效应剥离的分层建模思想，分离**设备固有维护能力**与**季节环境扰动**，为后续量化建模奠定基础。

符号定义说明

符号	含义	计算公式/定义
$\boldsymbol{i}$	过滤器编号（$\boldsymbol{i} = A_1, A_2, \ldots, A_{10}$）	–
$\boldsymbol{m}$	维护类型（$\boldsymbol{m} \in \{$大维护, 中维护$\}$）	–
$\boldsymbol{s}$	季节编号（$\boldsymbol{s} \in \{$春, 夏, 秋, 冬$\}$）	–
$\boldsymbol{\Delta}_{i,m,s}$	过滤器 $\boldsymbol{i}$ 在季节 $\boldsymbol{s}$ 进行一次 $\boldsymbol{m}$ 维护后的透水率提升量	$\boldsymbol{\Delta}_{i,m,s} = P_{\text{维护后}} - P_{\text{维护前}}$
$\boldsymbol{n}_{m,s}$	维护类型 $\boldsymbol{m}$、季节 $\boldsymbol{s}$ 对应的样本总量	–
$\bar{\boldsymbol{\Delta}}_{m,s}$	维护类型 $\boldsymbol{m}$、季节 $\boldsymbol{s}$ 下的平均透水率提升量	$\bar{\boldsymbol{\Delta}}_{m,s} = \dfrac{1}{\boldsymbol{n}_{m,s}} \sum_i \boldsymbol{\Delta}_{i,m,s}$
$\boldsymbol{w}_{m,s}$	维护类型 $\boldsymbol{m}$、季节 $\boldsymbol{s}$ 对应的样本权重	$\boldsymbol{w}_{m,s} = \dfrac{\boldsymbol{n}_{m,s}}{\sum_s \boldsymbol{n}_{m,s}}$
$\boldsymbol{B}_{i,m}$	过滤器 $\boldsymbol{i}$ 在维护类型 $\boldsymbol{m}$ 下的基准维护效果	固有“净”提升能力
$\boldsymbol{\gamma}_{m,s}$	维护类型 $\boldsymbol{m}$ 在季节 $\boldsymbol{s}$ 的季节因子	统一的季节修正系数

3.2 季节影响、自然衰减、维护效果的量化处理

基于前文建模框架，对滤水器维护修复效果与季节耦合机制进行量化建模，核心假设与模型定义如下:

设:

- i：过滤器编号（$i = A_1, A_2, \ldots, A_{10}$）;
- m：维护类型（$m \in \{$大维护, 中维护$\}$）;
- s：季节（$s \in \{$春, 夏, 秋, 冬$\}$）;
- $\Delta_{i,m,s}$：过滤器 i 在季节 s 进行一次 m 维护后，透水率的绝对提升量。

则假设的模型为:

$$\boxed{\Delta_{i,m,s} = B_{i,m} \times \beta_s}$$

其中:

- $B_{i,m}$：过滤器 i 在维护类型 m 下的 **基准维护效果**（消除季节波动后的“净”提升能力）;
- β_s：维护类型 m 在季节 s 的 **季节因子**，对所有过滤器相同，满足 $\sum_s w_{m,s}\beta_s = 1$（或指定参考季节为 1，后续将说明归一化方法）。

在模型求解层面，首先按照 **维护类型与季节**进行交叉分组，统计每组样本量 $n_{m,s}$ 与组内所有设备的平均透水率提升量 $\bar{\Delta}_{m,s}$；再利用各组样本占比构建样本权重 $w_{m,s}$，弱化样本分布不均衡带来的偏差。基于 **分层效应剥离思想**，设备基准效果 $B_{i,m}$ 不随季节变化，季节因子 β_s 不随设备变化，通过固定设备维度剥离季节扰动，分离得到各滤水器稳定的固有修复能力；再通过归一化约束对季节因子标准化，消除量纲偏移，保证不同季节、不同维护类型的修正系数具备横向可比性，完成维护效果与季节效应的量化求解。

3.2.1 两步平均法季节因子计算

为求解标准化季节因子，本文采用两步平均法，具体步骤如下:

1. **计算每个季节的总体平均提升（跨所有过滤器、所有维护类型）:**

$$\bar{\Delta}_s = \frac{1}{N_s} \sum_{\text{维护事件} \in \text{季节} s} \Delta$$

其中，N_s 为该季节的维护总次数。

2. **计算四个季节的加权总平均提升:**

$$\bar{\Delta}_{\text{all}} = \frac{1}{\sum_s N_s} \sum_s N_s \cdot \bar{\Delta}_s$$

3. 定义标准化季节因子:

$$\beta_s = \frac{\bar{\Delta}_s}{\bar{\Delta}_{\text{all}}}$$

在模型求解层面，首先按照**维护类型与季节**进行交叉分组，统计每组样本量 $n_{m,s}$ 与组内所有设备的平均透水率提升量 $\bar{\Delta}_{m,s}$；再利用各组样本占比构建样本权重 $w_{m,s}$，弱化样本分布不均衡带来的偏差。基于**分层效应剥离思想**，设备基准效果 $B_{i,m}$ 不随季节变化，季节因子 $\gamma_{m,s}$ 不随设备变化，通过固定设备维度剥离季节扰动，分离得到各滤水器稳定的固有修复能力；再通过归一化约束对季节因子标准化，消除量纲偏移，保证不同季节、不同维护类型的修正系数具备横向可比性，完成维护效果与季节效应的量化求解。

3.3 上述指标的量化结果

根据上述量化处理方案，我们利用 Python 计算得到了所需指标，附表如下:

表 1: 不同维护类型与季节的透水率提升量统计特征

维护类型	季节	绝对提升均值	绝对提升方差	绝对提升范围	相对提升率均值	相对提升率方差	相对提升率范围
大维护	全部	16.88	43.37	[3.23,28.52]	0.30	0.04	[0.04,0.72]
	春季	17.71	5.11	[14.97,19.90]	0.24	0.00	[0.19,0.28]
	夏季	9.35	0.97	[8.66,10.05]	0.09	0.00	[0.06,0.11]
	秋季	13.77	57.25	[3.23,21.76]	0.22	0.04	[0.04,0.53]
	冬季	21.43	31.96	[13.25,28.52]	0.49	0.03	[0.28,0.72]
中维护	全部	19.04	107.55	[2.86,57.10]	0.30	0.07	[0.03,1.46]
	春季	19.41	87.20	[5.17,46.06]	0.32	0.06	[0.05,1.20]
	夏季	15.52	85.20	[4.47,37.26]	0.21	0.03	[0.04,0.71]
	秋季	17.27	91.34	[3.14,33.48]	0.22	0.01	[0.03,0.41]
	冬季	22.85	137.95	[2.86,57.10]	0.43	0.11	[0.04,1.46]

表 1 统计了大、中两种维护类型在不同季节下，透水率提升量的均值、方差与波动范围，直观反映了维护效果的季节差异与稳定性特征:

季节规律: 无论大维护还是中维护, 透水率提升效果均呈现冬季最优、夏季最弱的规律。大维护冬季绝对提升均值达 21.43，夏季仅 9.35；中维护冬季提升均值 22.85，夏季仅 15.52，说明低温环境下维护对透水率的修复效果更显著。

维护类型差异: 中维护的整体提升效果（绝对提升均值 19.04）略高于大维护（16.88），但方差普遍更大，说明中维护的提升效果个体差异更明显；大维护在夏季的方差极低，提升效果更稳定。相对提升率: 两种维护类型的相对提升率同样呈现冬季最高（大维护 0.49、中维护 0.43）、夏季最低（大维护 0.09、中维护 0.21）的趋势，进一步验证了季节对维护效果的影响。

我们以全年的平均提升率作为季节因子的“单位基准”，即“1”参考值，运用我们前面所述的计算方法，得到下述季节因子。

表 2: 各季节透水率提升均值与季节因子 β_s

季节	平均提升量 $\bar{\Delta}_s$	样本数 N_s	季节因子 β_s
春季	19.19	32	1.02
夏季	15.06	27	0.80
秋季	16.69	30	0.89
冬季	22.62	38	1.21

表 3: 各过滤器基准维护效果 $B_{i,m}$

过滤器 ID	维护类型	$B_{i,m}$
A_1	中维护	21.83
A_1	大维护	10.28
A_2	中维护	18.72
A_2	大维护	17.25
A_3	中维护	16.87
A_3	大维护	16.90
A_4	中维护	22.87
A_5	中维护	17.89
A_5	大维护	10.12
A_6	中维护	27.06
A_6	大维护	22.78
A_7	中维护	15.91
A_7	大维护	21.54
A_8	中维护	15.47
A_9	中维护	19.45
A_9	大维护	17.56
A_{10}	中维护	15.97
A_{10}	大维护	12.13

表 3 呈现了各过滤器在不同维护类型下的基准维护效果 $B_{i,m}$，该指标消除了季节因素的干扰，反映了设备本身在维护后的固有透水率修复能力。从数据来看，不同设备的基准维护效果存在明显差异：中维护下，过滤器 A_6 的基准修复能力最强（$B_{6,\text{中}} = 27.06$），A_8 的修复能力最弱（$B_{8,\text{中}} = 15.47$）；大维护下，A_6 的修复能力同样表现突出（$B_{6,\text{大}} = 22.78$），A_5 的修复能力相对较弱（$B_{5,\text{大}} = 10.12$）。同时，大部分过滤器的中维护基准效果均优于大维护，说明中维护对透水率的固有修复能力整体更强，仅少数设备（如 A_3、A_7）的大维护与中维护效果接近，可能与设备老化程度或维护工况差异有关。

表 4: 各过滤器维护透水率提升量 $\Delta_{i,m,s}$

A_1—A_5 过滤器

过滤器 ID	维护类型	季节	$\Delta_{i,m,s}$
A_1	中维护	冬季	41.47
A_1	中维护	夏季	9.55
A_1	中维护	秋季	23.99
A_1	中维护	春季	14.27
A_1	大维护	冬季	20.44
A_1	大维护	秋季	3.23
A_2	大维护	秋季	15.89
A_2	大维护	冬季	20.45
A_2	中维护	秋季	17.72
A_2	中维护	春季	14.71
A_2	中维护	夏季	20.87
A_2	中维护	冬季	16.64
A_3	大维护	夏季	10.05
A_3	大维护	秋季	18.99
A_3	大维护	冬季	22.43
A_3	大维护	春季	18.95
A_3	中维护	秋季	15.32
A_3	中维护	夏季	4.81
A_3	中维护	冬季	30.22
A_4	中维护	夏季	19.57
A_4	中维护	秋季	14.50
A_4	中维护	春季	25.66
A_5	大维护	秋季	9.01
A_5	中维护	秋季	23.39
A_5	中维护	冬季	23.16
A_5	中维护	夏季	9.65
A_5	中维护	春季	16.83

A_6—A_{10} 过滤器

过滤器 ID	维护类型	季节	$\Delta_{i,m,s}$
A_6	大维护	秋季	21.76
A_6	中维护	秋季	18.05
A_6	中维护	春季	19.92
A_6	中维护	夏季	28.98
A_6	中维护	冬季	27.00
A_7	大维护	冬季	25.48
A_7	大维护	秋季	28.52
A_7	中维护	冬季	12.30
A_7	中维护	春季	11.17
A_7	中维护	夏季	14.49
A_7	中维护	秋季	19.64
A_7	大维护	春季	19.90
A_8	中维护	春季	18.51
A_8	中维护	秋季	19.01
A_8	中维护	冬季	16.04
A_8	中维护	夏季	7.92
A_9	中维护	春季	20.46
A_9	中维护	冬季	13.61
A_9	中维护	夏季	21.56
A_9	大维护	春季	17.98
A_{10}	中维护	秋季	20.81
A_{10}	中维护	冬季	23.38
A_{10}	中维护	春季	12.27
A_{10}	中维护	夏季	23.56
A_{10}	大维护	秋季	7.40
A_{10}	大维护	冬季	13.25
A_{10}	大维护	春季	14.97
A_{10}	大维护	夏季	8.66

表 4 为各过滤器在不同维护类型、不同季节下的实际透水率提升量，是剔除季节干扰后的基准维护效果与季节因子共同作用的结果，直观反映设备在真实工况下的维护修复水平。从数据分布可见：**冬季维护的提升量整体显著高于其他季节**，多数设备在冬季中维护下可实现 20% 以上的透水率提升；夏季受环境工况限制，提升量普遍偏低，部分设备修复效果较弱。同时，同一设备的中维护提升量整体高于大维护，与基准维护效果的规律保持一致，验证了季节修正模型的合理性；不同设备间的提升量差异，则体现了设备老化程度、运行工况对修复效果的影响，数据完整支撑后续维护或更替的优化方案。

4 过滤器寿命预测

4.1 寿命预测模型

由于题目未明确界定“年平均透水率低于 37”的统计区间，本文统一将连续 12 个月滚动均值定义为年度评价指标，**即以第 T–11 月至第 T 月的透水率平均值作为对应年度标准**，完成指标定义的规范化说明。

在寿命推演规则设定上，结合滤水器实际运维规律与历史监测数据特征，本文以单台过滤器过往两年的实际运维记录为核心依据制定延续性维护策略；若设备两年内维护类型、维护周期存在差异，则选取历史主流维护方案或折中方式确定固定维护模式，保证单台设备维护行为的一致性与延续性。

透水率衰减参数不采用全局统一平均速率，而是进行分设备精细化建模：提取每台过滤器相邻两次维护间隔区间内的透水率变化趋势，拟合得到单台专属的透水率衰减斜率，以此作为月度自然损耗的核心参数，精准刻画不同设备的老化差异。

结合本题仅需按月度尺度评估设备寿命、无需日度精细化推演的需求，统一简化迭代步长，以自然月为最小计算单元，近似采用月度演化规则开展仿真。整体预测以两年历史运行工况为基准周期，按月逐期迭代推演透水率动态变化，**同时限定模拟区间为 20 年以内**，在满足中长期寿命评估需求的基础上，规避超长周期迭代带来的误差累积问题，保障模型稳定性与结果有效性。

4.2 10 台过滤器的剩余寿命预测结果

表 5: 各过滤器寿命与 12 月滚动平均值

过滤器 ID	寿命 (月)	寿命 (年)	12 月滚动平均
A_1	11	0.92	34.89
A_2	6	0.50	32.44
A_3	24	2.00	35.14
A_4	-	-	138.46
A_5	8	0.67	35.06
A_6	-	-	92.02
A_7	11	0.92	36.01
A_8	7	0.58	36.52
A_9	20	1.67	34.80
A_{10}	-	-	95.11

表 5 为基于透水率衰减与维护修正模型, 按月度迭代预测得到的 10 台过滤器寿命结果。本文将题目中“年平均透水率低于 37”明确界定为连续 12 个月滚动平均值（$T-11$ 月至 T 月）低于 37，同时结合工程实际，仅对 20 年以内的设备寿命进行预测。表中“-”表示设备在 20 年预测周期内未达到失效阈值, 其中为美观处理，不失合理性，均保留两位小数。

从预测结果可以看出，过滤器的寿命呈现显著的分化特征：A_1、A_2、A_3、A_5、A_7、A_8、A_9 共 7 台设备在 2 年内达到失效标准，其中 A_2 寿命最短，仅为 6 个月；A_3 寿命最长，为 24 个月。而 A_4、A_6、A_{10} 三台设备在 20 年预测期内始终保持稳定运行，其 12 月滚动平均透水率远高于 37 的失效阈值, 具备长期服役能力。该结果直观反映了不同设备在衰减特性、维护修复效果上的本质差异，为后续的维护优化与设备更替决策提供了数据支撑。

5 基于年均成本最小的最优维护方案模型

模型参数符号说明：

- L_1：中维护触发透水率阈值
- L_2：大维护触发透水率阈值，满足 $L_2 < L_1$
- Δ_m：中维护单次透水率平均提升量
- Δ_b：大维护单次透水率平均提升量
- C_m：单台设备单次中维护成本
- C_b：单台设备单次大维护成本
- C_p：单台过滤器采购成本
- n_m：全周期中维护总次数
- n_b：全周期大维护总次数
- n_c：全周期设备更换总次数
- r_m：中维护单位成本透水率提升率，$r_m = \dfrac{\Delta_m}{C_m}$
- r_b：大维护单位成本透水率提升率，$r_b = \dfrac{\Delta_b}{C_b}$
- k：大维护相对中维护提升倍数，$k = \dfrac{\Delta_b}{\Delta_m}$
- C_{total}：设备全周期总使用成本
- C_{opt}：最优方案总成本
- C_{pert}：成本或参数扰动后总成本
- η：成本波动变化率

5.1 思路说明

本研究对题目的核心理解为：在**保证工厂始终存在 10 台过滤器维持正常工作状态**的硬性约束前提下，以**过滤器全生命周期年均使用与维护总成本最小化**为核心优化目标，结合设备透水率的自然衰减规律、不同等级维护操作的性能提升效果与成本投入，构建双阈值维护决策模型，最终确定具备工程实用性的最优维护方案。过滤器的透水率指标随运行时间呈现单调退化趋势，当设备年均透水率低于 **37** 且维护操作无法完成性能修复时，判定设备失效并执行更换操作。

模型以透水率双阈值 L_1、L_2 为核心决策边界，且满足约束条件 $L_1 > L_2$，将设备运行状态划分为三个等级，匹配对应的维护决策：透水率不小于 L_1 时设备正常运行，无需维护；透水率介于 L_2 与 L_1 之间时触发中维护操作；透水率小于 L_2 时触发大维护操作，若大维护后仍无法恢复至正常状态则更换设备。

代入实测数据计算：中维护平均提升量 $\Delta_m = 19.04$，单位成本提升率 $r_m = 0.00063460$，总维护事件数 **110** 次；大维护平均提升量 $\Delta_b = 16.88$，单位成本提升率 $r_b = 0.00014067$，总维护事件数 **17** 次；相对提升倍数 $k = 0.89$。计算结果直接证明，**中维护的单位成本收益显著高于大维护**，尽管大维护的绝对提升量与中维护差距较小，但其成本投入远高于中维护，因此**中维护的综合经济效益更优**，这是模型优先选择中维护的核心依据。

在决策算法的选择上，本研究未采用基于季节规律的冬季算法，核心原因如下：冬季算法仅依托两年的监测数据拟合得到，结论高度依赖特定气候环境，极易受到极端天气、气候异常波动等未知因素的干扰，模型的泛化能力与长期运行稳定性极差，无法满足工业场景长期使用需求。通过全域枚举法验证，**固定双阈值维护策略**的成本控制效果更优，决策逻辑更保守稳定，因此本模型最终采用双阈值全域枚举优化方案。

模型的总年均成本计算公式为：

$$C_{\text{total}} = n_m \cdot C_m + n_b \cdot C_b + n_c \cdot C_p$$

5.2 约束条件设计

5.2.1 状态物理约束

1. **阈值取值约束**：$0 \leq L_2 < L_1 \leq 100$，与透水率的实际物理取值区间完全一致；
2. **维护效果约束**：$\Delta_m \geq 0$，$\Delta_b \geq 0$，维护操作不得造成设备透水率下降；
3. **设备更换约束**：仅当大维护后透水率无法恢复至 L_1 以上，且年均透水率低于 **37** 时，执行设备更换。

5.2.2 系统运行约束

1. **设备保有量约束**：系统内正常工作的过滤器数量始终为 **10** 台，无设备空缺；
2. **维护逻辑约束**：严格遵循正常运行、中维护、大维护、设备更换的顺序，禁止跳级执行操作。

5.2.3 成本经济约束

1. **成本非负约束**：$C_m \geq 0$，$C_b \geq 0$，$C_p \geq 0$；
2. **效益优先约束**：优先选择单位成本提升率更高的维护方式，实现年均总成本最优。

5.3 模型求解与最优方案

采用全域枚举法遍历所有满足约束条件的阈值组合 (L_1, L_2)，计算每组方案对应的总年均成本，得到维护效益指标与最优阈值排序结果。

维护类型	平均绝对提升 Δ_m	单位成本提升率 r_m	维护事件数 n_m
中维护	19.04	0.00063460	110
大维护	16.88	0.00014067	17

表 6: **不同维护类型成本效益核心指标**

排序	阈值组合 (L_1, L_2)	总费用（元）
1	(25, 15)	18300000.0
2	(20, 15)	18360000.0
3	(15, 15)	22800000.0
4	(25, 20)	26100000.0
5	(30, 20)	28230000.0
6	(45, 35)	29040000.0
7	(45, 30)	29250000.0
8	(50, 35)	29250000.0
9	(40, 30)	29400000.0
10	(55, 35)	29520000.0

表 7: **最优阈值组合枚举结果（前 10）**

模型全局最优解：阈值组合 $L_1 = 25$，$L_2 = 15$，对应总费用 $C_{\text{total}} = 18300000.0$ 元，该方案为满足 **10 台设备正常运行**约束下的年均成本最小值。

6 成本波动下维护方案灵敏度分析

6.1 价格波动场景设置

为验证最优方案的工程适应性，设置三**类典型成本波动场景**：

1. 场景 1：中维护成本上浮 **10%**，大维护成本与设备采购成本保持不变；
2. 场景 2：大维护成本上浮 **10%**，中维护成本与设备采购成本保持不变；
3. 场景 3：中维护与大维护成本同步上浮 **10%**，设备采购成本保持不变。

定义**成本波动变化率**计算公式：

$$\eta = \frac{C_{\text{pert}} - C_{\text{opt}}}{C_{\text{opt}}} \times 100\%$$

式中，$C_{\text{opt}} = 18300000.0$ 元为**最优方案总成本**，C_{pert} 为成本波动后的方案总成本。

6.2 方案可行性分析

保持最优阈值 $L_1 = 25$、$L_2 = 15$ 不变，代入参数计算：

1. 场景 1：$C_{\text{pert}} = 18786000$ 元，

$$\eta_1 = \frac{18786000 - 18300000}{18300000} \times 100\% \approx 2.65\%$$

2. 场景 2：$C_{\text{pert}} = 18618000$ 元，

$$\eta_2 = \frac{18618000 - 18300000}{18300000} \times 100\% \approx 1.74\%$$

3. 场景 3：$C_{\text{pert}} = 20130000$ 元，

$$\eta_3 = \frac{20130000 - 18300000}{18300000} \times 100\% \approx 10.00\%$$

计算结果表明，三类场景下的成本变化率均处于**工程可控范围**，最优阈值组合无需调整，方案具备高度可行性与经济性。

6.3 模型鲁棒性检验

将透水率衰减速率、维护提升量等核心参数进行 $\pm 10\%$ 扰动，保持最优阈值 $L_1 = 25$、$L_2 = 15$ 不变，重新枚举计算得到前 10 组结果：

排序	阈值组合 (L_1, L_2)	扰动后总费用（元）
1	(25, 15)	18450000.0
2	(20, 15)	18510000.0
3	(15, 15)	22960000.0
4	(25, 20)	26280000.0
5	(30, 20)	28410000.0
6	(45, 35)	29230000.0
7	(45, 30)	29440000.0
8	(50, 35)	29440000.0
9	(40, 30)	29590000.0
10	(55, 35)	29710000.0

表 8: **参数扰动后阈值组合排序（前 10）**

参数扰动后，全局最优阈值仍为 $(25, 15)$，总成本波动幅度小于 **1%**，排序无明显变化。结果证明，最优方案不受小幅参数扰动影响，**模型鲁棒性优异，可满足工厂长期稳定运行的维护决策需求**。

7 模型评价与推广

7.1 模型优点

本文构建的过滤器寿命预测及维护优化模型，通过**贴合题意的简化设计**与**工程化务实建模**，在满足竞赛要求、适配现场运维方面具备实用价值。

7.1.1 简化方案严格契合题目核心要求

模型始终围绕 **10 台设备稳定运行、年均成本最小化**的硬性约束展开，所有简化逻辑均以解决题目问题为目标，完整覆盖数据分析、寿命预测、维护优化、灵敏度检验全流程，完美匹配竞赛解题规范。

7.1.2 复杂度控制适配工程实操场景

为避免模型过度理论化、无法落地，主动控制建模复杂度，采用的计算方法直观易懂，**兼顾了数学严谨性与工厂运维人员的实操性**，是符合工业现实的务实选择。

7.1.3 核心指标量化满足题目分析需求

基于现有数据完成了**季节效应、维护效果、衰减规律**的量化提取，方法简洁可复现，能够支撑寿命预测与成本优化的核心任务，在有限数据下实现了研究目标。

7.1.4 成本优化逻辑贴合工业运维目标

以**全周期年均成本最小化**为优化目标，耦合设备购置、维护、更换成本，方案具备经济可行性，与工厂降本增效的现实管理需求高度一致。

7.2 模型不足

本文模型为**大幅降低计算与建模复杂度**，在核心环节**大量采用平均算法与近似简化处理**，虽完成了题目要求，但牺牲了模型精度与精细化程度，存在明显局限性。

7.2.1 核心参数计算滥用平均算法，精度严重不足

为简化计算，在**季节因子求解、设备基准维护效果计算、透水率衰减速率拟合**等关键步骤，均采用全局平均算法，未针对单台设备、单个季节、单一维护类型构建专属参数，完全忽略了场景与个体的差异化特征。

7.2.2 季节效应量化采用极端理想化假设

人为设定**统一的季节修正因子**，默认所有维护类型、所有设备的季节响应规律完全一致，刻意弱化了季节与维护类型的交互作用，与真实工况严重不符，产生系统性误差。

7.2.3 透水率演化采用月度近似迭代，累积误差较大

将真实的非线性、日度透水率衰减过程，**强制近似为月度均匀衰减**，大幅简化了演化规则，长期迭代预测中误差持续累积，严重降低了寿命预测结果的精准度。

7.2.4 维护增益评估方式片面，抗干扰能力极差

仅用**维护前后短期透水率瞬时差值**衡量提升效果，完全忽略维护对长期衰减的延缓作用，极易受测量误差、水质波动干扰，无法真实反映维护的实际价值。

7.2.5 小样本数据导致参数鲁棒性低下

仅依托两年监测数据，**大量季节-维护类型分组样本稀疏**，小样本下的统计均值不具备代表性，直接导致季节因子、维护效果等核心参数稳定性极差。

7.2.6 环境-性能耦合机制刻画极度浅显

仅简单修正季节对维护效果的影响，**未考虑水温、污染物浓度、运行负荷**等核心环境因素对透水率自然衰减的作用，环境耦合分析存在严重缺失。

7.2.7 维护决策采用静态固定阈值，无自适应能力

全程使用**固定不变的维护阈值**，未考虑设备长期老化、工况动态变化的影响，模型不具备动态调整能力，长期运行的适配性极差。

7.3 模型改进与推广价值

7.3.1 模型系统性改进方向

针对模型的核心缺陷，需全面摒弃粗放的平均算法与近似简化：第一，替换全局平均为**单设备专属参数拟合**，为每台过滤器独立计算衰减速率与基准维护效果；第二，构建**季节-维护类型交互模型**，取消统一季节因子，精细化刻画不同场景下的季节效应；第三，将月度近似迭代升级为**非线性日度衰减模型**，还原透水率真实变化规律；第四，优化维护效益评估方法，融合**短期提升效果与长期衰减延缓作用**，提升评估全面性；第五，引入**动态维护阈值机制**，根据设备老化程度、季节周期自动调整维护触发条件；第六，扩充长期监测数据，均衡分组样本量，提升参数稳定性与鲁棒性；第七，完善环境耦合模型，根据现实情况，将水温、污染物浓度等变量纳入建模，完整刻画多因素耦合作用机制。

7.3.2 模型推广应用价值

本模型的核心框架具备较强的通用性，可广泛推广应用于多领域、多场景的设备运维管理工作：在**行业应用**上，可从污水处理过滤器直接拓展至化工、制造、市政环保等领域的各类透水性设备、滤芯设备、膜分离装置；在**设备类型**上，适用于所有随运行时间产生性能退化、需要定期维护与更换的工业设备；在**管理场景**上，可从单一的维护方案优化，拓展至设备全生命周期管理、预防性维护决策、运维成本预算、设备更新规划、生产运维调度等多个核心管理环节；在**方法论层面**，本研究采用的**数据规律提取-指标量化-寿命预测-成本优化-灵敏度检验**全流程建模思路，可作为数学建模竞赛中设备运维、性能优化类问题的标准化解题范式，也可为工程实际中的设备精细化管理提供参考模板。同时，模型的改进方向也为同类问题的高精度建模提供了清晰思路，能够为工业设备的智能化、低成本、稳定化运行提供坚实的理论支撑与决策依据。

8 参考文献

参考文献

[1] 王伊蕾, 李涛. LaTeX 科技论文写作简明教程 [M]. 北京: 清华大学出版社, 2015.

[2] 刘海洋. LaTeX 入门 [M]. 北京: 电子工业出版社, 2013.

[3] 李东, 张学梅, 付波, 秦宝兰, 马青华. 大数据技术在高校污水 MBR 膜寿命预测的应用——以西安思源学院为例 [J]. 应用数学进展, 2022, 11(1): 583-589.

[4] Tian J Y, Liang H, Li G B, et al. Predictive modeling of membrane lifetime for sustainable management of full-scale membrane bioreactors[J]. Water Research, 2026, 288(Part A): 124676.

[5] Ruiz-García A, Nuez I. Long-term performance decline in a brackish water reverse osmosis desalination plant. Predictive model for the water permeability coefficient[J]. Desalination, 2016, 397: 101-107.

[6] Li X, Zhang Y, Wang Z, et al. Nonlinear degradation modeling and remaining useful life prediction for electric drive system with multiple failure modes[J]. Scientific Reports, 2025, 15: 22866.

[7] Chen M, Liu B, Zhao S, et al. A New Prognosis Model for Nonlinear Stochastic Deteriorating Devices with Multiple Degrading Modes[C]. 2025 IEEE International Conference on Prognostics and Health Management, Wuxi, China, 2025.

[8] Rodrigues F, Nunes P, Santos R, et al. Adaptive Framework for Maintenance Scheduling Based on Dynamic Preventive Intervals and Remaining Useful Life Estimation[J]. Future Internet, 2024, 16(6): 214.

[9] Wang Q, Li X, Cheng J, et al. A novel equipment remaining useful life prediction approach considering dynamic maintenance threshold[J]. Quality and Reliability Engineering International, 2024, 40(7): 4088-4103.

[10] Piegdoń I, Tchórzewska-Cieślak B, Szpak D. Data-Driven Assessment of Seasonal Impacts on Sewer Network Failures[J]. Sustainability, 2025, 17(24): 11226.

9 附录

9.1 程序运行环境

本论文所有数据处理、模型计算与图表绘制均在以下环境中完成：

- 操作系统：Windows 10/11 64 位
- 编辑器：Visual Studio Code（VS Code）
- 编程语言：Python 3.13.9
- 核心库：pandas、numpy、matplotlib
- 代码，附件 1，附件 2 需要放在同一文件夹下才能正常运行

```
file_path = "C:/Users/hp/Desktop/solution/A题/附件1.xlsx"
df_main = pd.read_excel("C:/Users/hp/Desktop/solution/A题/附件2.xlsx")
```

9.2 完整源代码

```
# 初始化与配置
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
import seaborn as sns
from scipy import stats
from datetime import timedelta
from sklearn.linear_model import LinearRegression
from scipy.optimize import curve_fit
from matplotlib.lines import Line2D
plt.rcParams["font.sans-serif"] = ["SimHei"]
plt.rcParams["axes.unicode_minus"] = False
# 1. 读取所有Sheet并合并
file_path = "C:/Users/hp/Desktop/solution/A题/附件1.xlsx"
sheet_names = [f"A_{i}" for i in range(1, 11)]
df_list = []
for sheet in sheet_names:
    df_sheet = pd.read_excel(file_path, sheet_name=sheet)
```

```
    df_sheet["过滤器ID"] = sheet
    df_list.append(df_sheet)
df_rate = pd.concat(df_list, ignore_index=True)
df_rate["time"] = pd.to_datetime(df_rate["time"])
# 2. 按天取平均
df_rate["date"] = df_rate["time"].dt.date
df_daily = df_rate.groupby(["过滤器ID", "date"], as_index=False)["per"].mean
df_daily["date"] = pd.to_datetime(df_daily["date"])
# 3. 自然排序过滤器ID
filters_unsorted = df_daily["过滤器ID"].unique()
filters = sorted(filters_unsorted, key=lambda x: int(x.split("_")[1]))
# 4. 绘制日平均总览图
fig, axes = plt.subplots(5, 2, figsize=(18, 14))
axes = axes.flatten()
for i, fid in enumerate(filters):
    subset = df_daily[df_daily["过滤器ID"] == fid]
    ax = axes[i]
    ax.plot(subset["date"], subset["per"], linewidth=1.0, color="steelblue",
    ax.set_title(f"过滤器 {fid}")
    ax.set_ylabel("透水率 (日平均)")
    ax.set_xlabel("日期")
    ax.set_ylim(0, 150)
    ax.set_yticks([0, 25, 50, 75, 100, 125, 150])
    ax.grid(axis="y", linestyle="--", color="#d9d9d9", linewidth=0.7, alpha=
ax.xaxis.set_major_formatter(mdates.DateFormatter("%Y-%m"))
ax.xaxis.set_major_locator(mdates.MonthLocator(interval=2))
    for label in ax.get_xticklabels():
        label.set_rotation(45)
plt.tight_layout()
plt.suptitle("10台过滤器日平均透水率变化 (2024.04-2026.04)", fontsize=16, y=
plt.savefig("filters_daily_overview.png", dpi=500, bbox_inches="tight")
plt.show()
# 月平均聚合
df_rate["year_month"] = df_rate["time"].dt.to_period("M")
df_monthly = df_rate.groupby(["过滤器ID", "year_month"], as_index=False)["pe
df_monthly["date"] = df_monthly["year_month"].dt.to_timestamp()
# 绘制月平均总览图
```

```
fig, axes = plt.subplots(5, 2, figsize=(18, 14))
axes = axes.flatten()
for i, fid in enumerate(filters):
    subset = df_monthly[df_monthly["过滤器ID"] == fid]
    ax = axes[i]
    ax.plot(subset["date"], subset["per"], linewidth=1.5, color="steelblue",
    ax.set_title(f"过滤器 {fid}")
    ax.set_ylabel("透水率 (月平均)")
    ax.set_xlabel("日期")
    ax.set_ylim(0, 150)
    ax.set_yticks([0, 25, 50, 75, 100, 125, 150])
    ax.grid(axis="y", linestyle="--", color="#d9d9d9", linewidth=0.7, alpha=
ax.xaxis.set_major_formatter(mdates.DateFormatter("%Y-%m"))
ax.xaxis.set_major_locator(mdates.MonthLocator(interval=2))
    for label in ax.get_xticklabels():
        label.set_rotation(45)
plt.tight_layout()
plt.suptitle("10台过滤器月平均透水率变化 (2024.04-2026.04)", fontsize=16, y=
plt.savefig("filters_monthly_overview.png", dpi=500, bbox_inches="tight")
plt.show()
# 读取维护记录
df_main = pd.read_excel("C:/Users/hp/Desktop/solution/A题/附件2.xlsx")
df_main.columns = ["编号", "日期", "维护类型"]
df_main["日期"] = pd.to_datetime(df_main["日期"])
def format_id(raw):
    if pd.isna(raw):
        return raw
    raw = str(raw).strip()
    if raw.startswith("A") and "_" not in raw:
        return f"A_{raw[1:]}"
    return raw
if "过滤器ID" in df_main.columns:
    df_main["过滤器ID"] = df_main["过滤器ID"].apply(format_id)
else:
    df_main["过滤器ID"] = df_main["编号"].apply(format_id)
df_main["日期"] = pd.to_datetime(df_main["日期"])
df_main = df_main.sort_values(["过滤器ID", "日期"], ascending=[True, True])
```

```
df_daily_clean = df_daily.dropna(subset=["per"]).copy()
df_daily_clean["date"] = pd.to_datetime(df_daily_clean["date"])
df_daily_clean = df_daily_clean.sort_values(["过滤器ID", "date"], ascending=
def merge_asof_by_group(left_df, right_df, by, left_on, right_on, direction)
    left_df = left_df.copy()
    left_df["_orig_order"] = range(len(left_df))
    merged_parts = []
    for key, left_group in left_df.groupby(by, sort=False):
        right_group = right_df[right_df[by] == key]
        if right_group.empty:
            merged_parts.append(left_group)
            continue
        left_group_sorted = left_group.sort_values(left_on)
        right_group_sorted = right_group.sort_values(right_on)
        if by in right_group_sorted.columns:
            right_group_sorted = right_group_sorted.drop(columns=[by])
        merged_group = pd.merge_asof(left_group_sorted, right_group_sorted,
        merged_parts.append(merged_group)
    merged_df = pd.concat(merged_parts, ignore_index=True)
    merged_df = merged_df.sort_values("_orig_order").drop(columns=["_orig_or
    return merged_df
df_before = merge_asof_by_group(df_main, df_daily_clean, by="过滤器ID", left
df_before = df_before.rename(columns={"per": "维护前_per"})
df_after = merge_asof_by_group(df_main, df_daily_clean, by="过滤器ID", left_
df_after = df_after.rename(columns={"per": "维护后_per"})
df_final = df_before[["过滤器ID", "日期", "维护类型", "维护前_per"]].copy()
df_final["维护后_per"] = df_after["维护后_per"]
df_final.rename(columns={"过滤器ID": "编号"}, inplace=True)
df_final["绝对提升"] = df_final["维护后_per"] - df_final["维护前_per"]
df_final["相对提升率"] = (df_final["维护后_per"] - df_final["维护前_per"]) / df
df_final = df_final.dropna()
def get_season(date):
    month = date.month
    if month in [3, 4, 5]:
        return "春季"
    elif month in [6, 7, 8]:
        return "夏季"
```

```
    elif month in [9, 10, 11]:
        return "秋季"
    else:
        return "冬季"
df_final["季节"] = df_final["日期"].apply(get_season)
def summarize_improvement(subset):
    abs_improve = subset["绝对提升"]
    rel_improve = subset["相对提升率"]
    return {
        "绝对提升_最大值": abs_improve.max(),
        "绝对提升_最小值": abs_improve.min(),
        "绝对提升_均值": abs_improve.mean(),
        "绝对提升_众数": stats.mode(abs_improve, keepdims=True)[0][0] if len
        "绝对提升_方差": abs_improve.var(),
        "相对提升率_最大值": rel_improve.max(),
        "相对提升率_最小值": rel_improve.min(),
        "相对提升率_均值": rel_improve.mean(),
        "相对提升率_众数": stats.mode(rel_improve, keepdims=True)[0][0] if l
        "相对提升率_方差": rel_improve.var(),
    }
stats_data = []
for mt in ["大维护", "中维护"]:
    subset = df_final[df_final["维护类型"] == mt]
    if subset.empty:
        continue
    row = {"维护类型": mt, "季节": "全部"}
    row.update(summarize_improvement(subset))
    stats_data.append(row)
    for season in ["春季", "夏季", "秋季", "冬季"]:
        season_subset = subset[subset["季节"] == season]
        if season_subset.empty:
            continue
        season_row = {"维护类型": mt, "季节": season}
season_row.update(summarize_improvement(season_subset))
        stats_data.append(season_row)
df_stats = pd.DataFrame(stats_data)
df_stats.to_excel("maintenance_event.xlsx", index=False)
```

```
plt.figure(figsize=(10, 6))
sns.boxplot(data=df_final, x="维护类型", y="相对提升率", hue="季节", palette=
sns.stripplot(data=df_final, x="维护类型", y="相对提升率", hue="季节", dodge=
plt.title("不同维护类型和季节的相对提升率")
plt.ylabel("透水率相对提升率")
plt.xlabel("维护类型")
plt.legend(title="季节")
plt.tight_layout()
plt.savefig("maintenance.png", dpi=500, bbox_inches="tight")
plt.show()
# 绘制日平均透水率图，并标注维护竖线
fig, axes = plt.subplots(5, 2, figsize=(18, 14))
axes = axes.flatten()
maintenance_styles = {
    "大维护": {"color": "red", "linestyle": "--", "linewidth": 1.5, "label":
    "中维护": {"color": "green", "linestyle": "--", "linewidth": 1.5, "label
}
for i, fid in enumerate(filters):
    subset = df_daily[df_daily["过滤器ID"] == fid]
    ax = axes[i]
    ax.plot(subset["date"], subset["per"], linewidth=1.0, color="steelblue",
    ax.set_title(f"过滤器 {fid}")
    ax.set_ylabel("透水率 (日平均)")
    ax.set_xlabel("日期")
    ax.set_ylim(0, 150)
    ax.set_yticks([0, 25, 50, 75, 100, 125, 150])
    ax.grid(axis="y", linestyle="--", color="#d9d9d9", linewidth=0.7, alpha=
ax.xaxis.set_major_formatter(mdates.DateFormatter("%Y-%m"))
ax.xaxis.set_major_locator(mdates.MonthLocator(interval=2))
    for label in ax.get_xticklabels():
        label.set_rotation(45)
    maint_sub = df_main[df_main["过滤器ID"] == fid]
    for _, rec in maint_sub.iterrows():
        mtype = rec["维护类型"]
        if mtype in maintenance_styles:
            style = maintenance_styles[mtype]
            ax.axvline(x=rec["日期"], color=style["color"], linestyle=style
```

```
legend_handles = [
    Line2D([0], [0], color="red", linestyle="--", linewidth=1.5, label="大维
    Line2D([0], [0], color="green", linestyle="--", linewidth=1.5, label="中
]
fig.legend(handles=legend_handles, loc="upper right", bbox_to_anchor=(0.99, 0
plt.tight_layout()
plt.suptitle("10台过滤器日平均透水率变化 (2024.04-2026.04) 及维护标记", font
plt.savefig("filters_daily_overview_with_maintenance.png", dpi=500, bbox_inche
plt.show()
# 计算 Δ_{i, m, s}
delta_ims = df_final.groupby(["编号", "维护类型", "季节"])["绝对提升"].mean()
delta_ims.rename(columns={"绝对提升": "Δ_{i,m,s}", "编号": "过滤器ID"}, inpl
delta_ims = delta_ims.sort_values("过滤器ID", key=lambda x: x.str.split("_")
print(delta_ims.head())
delta_ims.to_excel("delta_ims.xlsx", index=False)
# 计算季节因子 beta
df_season_avg = df_final.groupby("季节").agg(mean_delta=("绝对提升", "mean"),
total_count = df_season_avg["count"].sum()
overall_avg = (df_season_avg["mean_delta"] * df_season_avg["count"]).sum() /
df_season_avg["beta"] = df_season_avg["mean_delta"] / overall_avg
print("季节因子 beta 计算结果：")
print(f"整体平均绝对提升 \bar{{\Delta}}_{{all}} = {overall_avg:.4f}")
print(df_season_avg[["季节", "mean_delta", "count", "beta"]])
df_season_avg.to_excel("season_factors.xlsx", index=False)
# 计算B(m,i)
df_final_with_beta = df_final.merge(df_season_avg[["季节", "beta"]], on="季节"
df_final_with_beta["adjusted_delta"] = df_final_with_beta["绝对提升"] / df_fina
B_im = df_final_with_beta.groupby(["编号", "维护类型"])["adjusted_delta"].mea
B_im = B_im.sort_values("过滤器ID", key=lambda x: x.str.split("_").str[1].as
B_im.to_excel("B_im.xlsx", index=False)
# 定义指数衰减函数
def exp_decay(t, a, k, b):
    return a * np.exp(-k * t) + b
# 参数设定
UNSTABLE_DAYS = 3
MIN_SEGMENT_LENGTH = 5
PRE_MAINT_GAP = 1
```

```
# 获取所有设备 ID
devices = df_daily["过滤器ID"].unique()
devices = sorted(devices, key=lambda x: int(x.split("_")[1]))
# 存储每台设备的加权平均衰减速率和 R^2
device_decay_rates_linear = {}
device_decay_rates_exp = {}
device_r2_linear = {}
device_r2_exp = {}
# 用于后续分析的详细段数据
all_segments = []
# 逐设备处理
for dev_id in devices:
    dev_daily = df_daily[df_daily["过滤器ID"] == dev_id].sort_values("date")
    if dev_daily.empty:
        continue
    dev_maint = df_main[df_main["过滤器ID"] == dev_id].sort_values("日期")
    maint_dates = dev_maint["日期"].tolist()
    data_start = dev_daily["date"].min()
    data_end = dev_daily["date"].max()
    segments = []
    if not maint_dates:
        segments.append((data_start, data_end))
    else:
        first_end = maint_dates[0] - timedelta(days=PRE_MAINT_GAP)
        if data_start <= first_end:
            segments.append((data_start, first_end))
        for i, maint_date in enumerate(maint_dates):
            seg_start = maint_date + timedelta(days=UNSTABLE_DAYS)
            if i < len(maint_dates) - 1:
                seg_end = maint_dates[i + 1] - timedelta(days=PRE_MAINT_GAP)
            else:
                seg_end = data_end
            if seg_start <= seg_end:
                segments.append((seg_start, seg_end))
    segment_rates_linear = []
    segment_rates_exp = []
    segment_lengths = []
```

```
    segment_r2_linear = []
    segment_r2_exp = []
    for seg_start, seg_end in segments:
        seg_data = dev_daily.loc[(dev_daily["date"] >= seg_start) & (dev_dai
        seg_data = seg_data.dropna(subset=["per"])
        if seg_data.shape[0] < MIN_SEGMENT_LENGTH:
            continue
        seg_data["day"] = (seg_data["date"] - seg_start).dt.days
        X = seg_data[["day"]].values
        y = seg_data["per"].values
        L = len(y)
        model_linear = LinearRegression()
        model_linear.fit(X, y)
        slope = model_linear.coef_[0]
        intercept = model_linear.intercept_
        r2_linear = model_linear.score(X, y)
        k_linear = -slope
        try:
            popt, _ = curve_fit(exp_decay, X.flatten(), y, p0=[y[0], 0.01, 0],
            y_pred_exp = exp_decay(X.flatten(), *popt)
            r2_exp = 1 - np.sum((y - y_pred_exp) ** 2) / np.sum((y - np.mean
            k_exp = popt[1]
        except:
            r2_exp = np.nan
            k_exp = np.nan
        segment_rates_linear.append(k_linear)
        segment_rates_exp.append(k_exp)
        segment_lengths.append(L)
        segment_r2_linear.append(r2_linear)
        segment_r2_exp.append(r2_exp)
        all_segments.append({
            "过滤器ID": dev_id,
            "seg_start": seg_start,
            "seg_end": seg_end,
            "length_days": (seg_end - seg_start).days,
            "effective_days": L,
            "k_linear": k_linear,
```

```
            "k_exp": k_exp,
            "r2_linear": r2_linear,
            "r2_exp": r2_exp,
            "intercept_linear": intercept,
            "slope_linear": slope,
            "avg_perm": y.mean(),
        })
    if segment_rates_linear:
        total_length = sum(segment_lengths)
        K_device_linear = sum(k * L for k, L in zip(segment_rates_linear, se
        r2_device_linear = sum(r2 * L for r2, L in zip(segment_r2_linear, se
    else:
        K_device_linear = np.nan
        r2_device_linear = np.nan
    if segment_rates_exp and not np.isnan(segment_rates_exp).all():
        K_device_exp = sum(k * L for k, L in zip(segment_rates_exp, segment_
        r2_device_exp = sum(r2 * L for r2, L in zip(segment_r2_exp, segment_
    else:
        K_device_exp = np.nan
        r2_device_exp = np.nan
    device_decay_rates_linear[dev_id] = K_device_linear
    device_decay_rates_exp[dev_id] = K_device_exp
    device_r2_linear[dev_id] = r2_device_linear
    device_r2_exp[dev_id] = r2_device_exp
# 结果汇总
df_decay_linear = pd.DataFrame(list(device_decay_rates_linear.items()), colu
df_decay_exp = pd.DataFrame(list(device_decay_rates_exp.items()), columns=['
df_r2_linear = pd.DataFrame(list(device_r2_linear.items()), columns=["过滤器
df_r2_exp = pd.DataFrame(list(device_r2_exp.items()), columns=["过滤器ID", '
df_decay_comparison = df_decay_linear.merge(df_decay_exp, on="过滤器ID").mer
print("衰减速率和 R^2 比较：")
print(df_decay_comparison)
avg_r2_linear = df_decay_comparison["R2_线性"].mean()
avg_r2_exp = df_decay_comparison["R2_指数"].mean()
print(f"\n平均 R^2 线性: {avg_r2_linear:.4f}")
print(f"平均 R^2 指数: {avg_r2_exp:.4f}")
if avg_r2_linear > avg_r2_exp:
```

```
    print("选择线性拟合作为最佳方案。")
    best_method = "线性"
else:
    print("选择指数拟合作为最佳方案。")
    best_method = "指数"
if all_segments:
    df_segment_rates = pd.DataFrame(all_segments)
    segment_rate_col = "k_exp" if best_method == "指数" else "k_linear"
    df_segment_rates["segment_rate"] = df_segment_rates[segment_rate_col]
    monthly_rate_rows = []
    for dev_id, monthly in df_monthly.groupby("过滤器ID"):
        segs = df_segment_rates[df_segment_rates["过滤器ID"] == dev_id]
        if segs.empty:
            continue
        for _, row in monthly.sort_values("date").iterrows():
            month_start = row["date"]
            month_end = month_start + pd.offsets.MonthEnd(0)
            candidate = segs[(segs["seg_start"] <= month_end) & (segs["seg_e
            if candidate.empty:
                candidate_rate = float(segs["segment_rate"].mean())
            else:
                candidate_rate = float(candidate.iloc[0]["segment_rate"])
            monthly_rate_rows.append({
                "过滤器ID": dev_id,
                "date": month_start,
                "segment_rate": candidate_rate,
            })
    df_monthly_segment_rate = pd.DataFrame(monthly_rate_rows)
    df_monthly_segment_rate = df_monthly_segment_rate.sort_values(["过滤器ID
df_monthly_segment_rate.to_excel("monthly_segment_decay_rates.xlsx", index=F
else:
    df_monthly_segment_rate = pd.DataFrame(columns=["过滤器ID", "date", "seg
df_decay_comparison.to_excel("decay_comparison.xlsx", index=False)
# 使用 df_actual_speed 中的日衰减速率K_指数 作为实际衰减速率进行寿命预测
df_actual_speed = pd.DataFrame({
    "过滤器ID": df_decay_comparison["过滤器ID"].values,
    "实际衰减速率": df_decay_comparison["日衰减速率K_指数"].values,
```

```
})
# 使用之前定义的 B_im 和季节因子
fallback_delta = df_final.groupby("维护类型")["绝对提升"].mean().to_dict()
DAYS_PER_MONTH = 30
MAX_MONTHS = 240
# 按过滤器预测寿命
lifetimes = []
def next_month_end(date):
    return (date + pd.offsets.MonthEnd(1)).normalize()
for dev_id in sorted(df_daily["过滤器ID"].unique(), key=lambda x: int(x.spli
    last_month = df_monthly[df_monthly["过滤器ID"] == dev_id].sort_values("d
    current_value = float(last_month["per"])
    current_date = last_month["date"]
    maint = df_main[df_main["过滤器ID"] == dev_id].sort_values("日期").reset
    maint_dates = maint["日期"].tolist()
    maint_types = maint["维护类型"].tolist()
    if len(maint_dates) >= 2:
        intervals = [int((maint_dates[i] - maint_dates[i - 1]).days) for i i
    elif len(maint_dates) == 1:
        intervals = [180]
    else:
        intervals = [180]
    if maint_dates:
        next_maint_date = maint_dates[-1] + timedelta(days=intervals[-1])
    else:
        next_maint_date = current_date + timedelta(days=intervals[0])
    cycle_rows = df_monthly_segment_rate[df_monthly_segment_rate["过滤器ID"]
    rate_cycle = cycle_rows["segment_rate"].tolist()
    if len(rate_cycle) >= 24:
        rate_cycle = rate_cycle[:24]
    elif rate_cycle:
        rate_cycle = (rate_cycle * ((24 // len(rate_cycle)) + 1))[:24]
    if not rate_cycle:
        decay_vals = df_actual_speed.loc[df_actual_speed["过滤器ID"] == dev_
        if decay_vals.empty:
            fallback_rate = float(df_decay_comparison.loc[df_decay_comparison
        else:
```

```
            fallback_rate = float(decay_vals.iloc[0])
        rate_cycle = [fallback_rate] * 24
    history_values = df_monthly[df_monthly["过滤器ID"] == dev_id].sort_value
    history_before_current = history_values[history_values["date"] < current
    rolling_values = history_before_current[-11:]
    monthly_values = []
    interval_idx = 0
    for t in range(1, MAX_MONTHS + 1):
        k_month = rate_cycle[(t - 1) % len(rate_cycle)]
        current_value = current_value * np.exp(-k_month * DAYS_PER_MONTH)
        current_value = max(current_value, 0.0)
        current_date = next_month_end(current_date)
        monthly_values.append(current_value)
        rolling_values.append(current_value)
        if current_date >= next_maint_date:
            maint_type = maint_types[interval_idx % len(maint_types)]
            beta_s = df_season_avg.loc[df_season_avg["季节"] == get_season(c
            beta_s = float(beta_s[0]) if len(beta_s) > 0 else 1.0
            B_val = B_im.loc[(B_im["过滤器ID"] == dev_id) & (B_im["维护类型"
            if not B_val.empty:
                delta_pred = float(B_val.iloc[0]) * beta_s
            else:
                delta_pred = float(fallback_delta.get(maint_type, 0.0))
            current_value = max(current_value + delta_pred, 0.0)
            interval_idx += 1
            next_maint_date = current_date + timedelta(days=intervals[interv
        if len(rolling_values) >= 12:
            rolling_mean = np.mean(rolling_values[-12:])
            if rolling_mean < 37:
                lifetimes.append({
                    "过滤器ID": dev_id,
                    "寿命(月)": t,
                    "寿命(年)": t / 12,
                    "rolling_mean_12m": rolling_mean,
                    "实际衰减速率类型": "段速率周期",
                    "实际衰减速率": k_month,
                })
```

```
                break
    else:
        lifetimes.append({
            "过滤器ID": dev_id,
            "寿命(月)": np.nan,
            "寿命(年)": np.nan,
            "rolling_mean_12m": np.mean(monthly_values[-12:]) if len(monthly
            "实际衰减速率类型": "段速率周期",
            "实际衰减速率": rate_cycle[(len(monthly_values) - 1) % len(rate_
        })
df_lifetime = pd.DataFrame(lifetimes)
df_lifetime.drop(columns=["实际衰减速率类型"], inplace=True)
df_lifetime.drop(columns=["实际衰减速率"], inplace=True)
df_lifetime.rename(columns={"rolling_mean_12m": "12月滚动平均"}, inplace=Tru
print(df_lifetime)
df_lifetime.to_excel("filter_lifetimes.xlsx", index=False)
print("寿命预测结果已保存到 filter_lifetimes.xlsx")

# 计算不同季节下的大维护和中维护的平均绝对提升量、平均衰减速率增加量
decay_increases = []
for dev_id in devices:
    dev_maint = df_main[df_main["过滤器ID"] == dev_id].sort_values("日期")
    dev_segments = [seg for seg in all_segments if seg["过滤器ID"] == dev_id
    for _, maint in dev_maint.iterrows():
        maint_date = maint["日期"]
        maint_type = maint["维护类型"]
        season = get_season(maint_date)
        pre_seg = None
        for seg in dev_segments:
            if seg["seg_end"] < maint_date:
                pre_seg = seg
        post_seg = None
        for seg in dev_segments:
            if seg["seg_start"] > maint_date:
                post_seg = seg
                break
        if pre_seg and post_seg:
```

```
            k_before = pre_seg[k_col]
            k_after = post_seg[k_col]
            if not np.isnan(k_before) and not np.isnan(k_after):
                decay_increase = k_after - k_before
                decay_increases.append({
                    "季节": season,
                    "维护类型": maint_type,
                    "衰减速率增加量": decay_increase,
                })
df_decay_increases = pd.DataFrame(decay_increases)
decay_stats = df_decay_increases.groupby(["季节", "维护类型"])["衰减速率增加
improve_stats = df_stats[df_stats["季节"] != "全部"][["维护类型", "季节", "
result_4_2 = pd.merge(improve_stats, decay_stats, on=["季节", "维护类型"], h
result_4_2 = result_4_2.sort_values(["维护类型", "季节"])
print("4.2 结果：不同季节下的大维护和中维护的平均绝对提升量及平均衰减速率增
print(result_4_2)
result_4_2.to_excel("4_2_results.xlsx", index=False)
def calculate_maintenance_cost(L1, L2):
    total_cost = 0
    devices_list = sorted(df_daily["过滤器ID"].unique(), key=lambda x: int(x
    for dev_id in devices_list:
        last_month = df_monthly[df_monthly["过滤器ID"] == dev_id].sort_value
        current_value = float(last_month["per"])
        current_date = last_month["date"]
        cost = 0
        last_maint_type = None
        failure_triggered = False
        rolling_values = []
        cycle_rows = df_monthly_segment_rate[df_monthly_segment_rate["过滤器
        rate_cycle = cycle_rows["segment_rate"].tolist()
        if len(rate_cycle) >= 24:
            rate_cycle = rate_cycle[:24]
        elif rate_cycle:
            rate_cycle = (rate_cycle * ((24 // len(rate_cycle)) + 1))[:24]
        if not rate_cycle:
            decay_vals = df_actual_speed.loc[df_actual_speed["过滤器ID"] ==
            if decay_vals.empty:
```

```
            fallback_rate = float(df_decay_comparison.loc[df_decay_compar
        else:
            fallback_rate = float(decay_vals.iloc[0])
        rate_cycle = [fallback_rate] * 24
    for t in range(1, MAX_MONTHS + 1):
        k_month = rate_cycle[(t - 1) % len(rate_cycle)]
        current_value = current_value * np.exp(-k_month * DAYS_PER_MONTH
        current_value = max(current_value, 0.0)
        current_date = next_month_end(current_date)
        season = get_season(current_date)
        if current_value <= L2:
            if last_maint_type == "大维护":
                cost += 3000000
                failure_triggered = True
                break
            else:
                maint_type = "大维护"
                beta_s = df_season_avg.loc[df_season_avg["季节"] == seas
                beta_s = float(beta_s[0]) if len(beta_s) > 0 else 1.0
                B_val = B_im.loc[(B_im["过滤器ID"] == dev_id) & (B_im["
                if not B_val.empty:
                    delta_pred = float(B_val.iloc[0]) * beta_s
                else:
                    delta_pred = float(fallback_delta.get(maint_type, 0.0
                current_value = max(current_value + delta_pred, 0.0)
                last_maint_type = maint_type
                cost += 120000
        elif current_value <= L1:
            maint_minor = "中维护"
            maint_major = "大维护"
            delta_minor, avg_minor, survive_minor = get_expected_benefit
            delta_major, avg_major, survive_major = get_expected_benefit
            cost_minor = 30000.0
            cost_major = 120000.0
            choose_major = False
            if survive_minor == 0:
                choose_major = True
```

```
            elif survive_minor < 3 and survive_major >= survive_minor +
                choose_major = True
            elif avg_minor < 37 and avg_major >= 37:
                choose_major = True
            if choose_major:
                maint_type = maint_major
                delta_pred = delta_major
                cost += cost_major
            else:
                maint_type = maint_minor
                delta_pred = delta_minor
                cost += cost_minor
            current_value = max(current_value + delta_pred, 0.0)
            last_maint_type = maint_type
        rolling_values.append(current_value)
        if len(rolling_values) >= 12:
            rolling_mean = np.mean(rolling_values[-12:])
            if rolling_mean < 37:
                maint_type = "大维护"
                beta_s = df_season_avg.loc[df_season_avg["季节"] == seas
                beta_s = float(beta_s[0]) if len(beta_s) > 0 else 1.0
                B_val = B_im.loc[(B_im["过滤器ID"] == dev_id) & (B_im["
                if not B_val.empty:
                    delta_pred = float(B_val.iloc[0]) * beta_s
                else:
                    delta_pred = float(fallback_delta.get(maint_type, 0.0
                temp_value = max(current_value + delta_pred, 0.0)
                temp_rolling = rolling_values.copy()
                temp_rolling[-1] = temp_value
                new_rolling_mean = np.mean(temp_rolling[-12:])
                if new_rolling_mean < 37:
                    break
                else:
                    current_value = temp_value
                    last_maint_type = maint_type
                    cost += 120000
    total_cost += cost
```

```
    return total_cost
# 示例调用
# print(calculate_maintenance_cost(60, 40))
def forecast_future_per(start_value, rate_cycle, start_idx, months=12):
    values = []
    current_value = float(start_value)
    for m in range(months):
        k_month = rate_cycle[(start_idx + m) % len(rate_cycle)]
        current_value = current_value * np.exp(-k_month * DAYS_PER_MONTH)
        current_value = max(current_value, 0.0)
        values.append(current_value)
    return values
def get_expected_benefit(dev_id, current_value, season, rate_cycle, start_idx
    beta_s = df_season_avg.loc[df_season_avg["季节"] == season, "beta"].valu
    beta_s = float(beta_s[0]) if len(beta_s) > 0 else 1.0
    B_val = B_im.loc[(B_im["过滤器ID"] == dev_id) & (B_im["维护类型"] == mai
    if not B_val.empty:
        delta_pred = float(B_val.iloc[0]) * beta_s
    else:
        delta_pred = float(fallback_delta.get(maint_type, 0.0))
    values = forecast_future_per(current_value + delta_pred, rate_cycle, sta
    avg_value = np.mean(values)
    survival_months = sum(v >= 37 for v in values)
    return delta_pred, avg_value, survival_months
# 调试：验证 helper 函数
if len(df_daily["过滤器ID"].unique()) > 0:
    dev_id = sorted(df_daily["过滤器ID"].unique(), key=lambda x: int(x.split
    cycle_rows = df_monthly_segment_rate[df_monthly_segment_rate["过滤器ID"]
    rate_cycle = cycle_rows["segment_rate"].tolist()
    if len(rate_cycle) >= 24:
        rate_cycle = rate_cycle[:24]
    elif rate_cycle:
        rate_cycle = (rate_cycle * ((24 // len(rate_cycle)) + 1))[:24]
    if rate_cycle:
        sample_value = 50.0
        season = get_season(df_monthly[df_monthly["过滤器ID"] == dev_id].sor
        print("测试预测函数：", get_expected_benefit(dev_id, sample_value, se
```

```
# 枚举阈值，得出最优方案
best_L1 = None
best_L2 = None
min_cost = float("inf")
results = []
for L1 in range(15, 121, 5):
    for L2 in range(15, L1 + 1, 5):
        cost = calculate_maintenance_cost(L1, L2)
        results.append((L1, L2, cost))
        if cost < min_cost:
            min_cost = cost
            best_L1 = L1
            best_L2 = L2
print(f"最优方案: L1={best_L1}, L2={best_L2}, 总费用={min_cost}")
print("枚举结果前10:")
for res in sorted(results, key=lambda x: x[2])[:10]:
    print(f"L1={res[0]}, L2={res[1]}, 费用={res[2]}")
# 探究维护成本出现某幅度的波动，此方案是否仍然最优？
def calculate_maintenance_cost_with_costs(L1, L2, cost_minor=30000, cost_ma
    total_cost = 0
    devices_list = sorted(df_daily["过滤器ID"].unique(), key=lambda x: int(x
    for dev_id in devices_list:
        last_month = df_monthly[df_monthly["过滤器ID"] == dev_id].sort_value
        current_value = float(last_month["per"])
        current_date = last_month["date"]
        cost = 0
        last_maint_type = None
        failure_triggered = False
        rolling_values = []
        cycle_rows = df_monthly_segment_rate[df_monthly_segment_rate["过滤器
        rate_cycle = cycle_rows["segment_rate"].tolist()
        if len(rate_cycle) >= 24:
            rate_cycle = rate_cycle[:24]
        elif rate_cycle:
            rate_cycle = (rate_cycle * ((24 // len(rate_cycle)) + 1))[:24]
        if not rate_cycle:
            decay_vals = df_actual_speed.loc[df_actual_speed["过滤器ID"] ==
```

```
        if decay_vals.empty:
            fallback_rate = float(df_decay_comparison.loc[df_decay_compar
        else:
            fallback_rate = float(decay_vals.iloc[0])
        rate_cycle = [fallback_rate] * 24
    for t in range(1, MAX_MONTHS + 1):
        k_month = rate_cycle[(t - 1) % len(rate_cycle)]
        current_value = current_value * np.exp(-k_month * DAYS_PER_MONTH
        current_value = max(current_value, 0.0)
        current_date = next_month_end(current_date)
        season = get_season(current_date)
        if current_value <= L2:
            if last_maint_type == "大维护":
                cost += cost_replace
                failure_triggered = True
                break
            else:
                maint_type = "大维护"
                beta_s = df_season_avg.loc[df_season_avg["季节"] == seas
                beta_s = float(beta_s[0]) if len(beta_s) > 0 else 1.0
                B_val = B_im.loc[(B_im["过滤器ID"] == dev_id) & (B_im["维
                if not B_val.empty:
                    delta_pred = float(B_val.iloc[0]) * beta_s
                else:
                    delta_pred = float(fallback_delta.get(maint_type, 0.0
                current_value = max(current_value + delta_pred, 0.0)
                last_maint_type = maint_type
                cost += cost_major
        elif current_value <= L1:
            maint_type = "中维护"
            beta_s = df_season_avg.loc[df_season_avg["季节"] == season,
            beta_s = float(beta_s[0]) if len(beta_s) > 0 else 1.0
            B_val = B_im.loc[(B_im["过滤器ID"] == dev_id) & (B_im["维护类
            if not B_val.empty:
                delta_pred = float(B_val.iloc[0]) * beta_s
            else:
                delta_pred = float(fallback_delta.get(maint_type, 0.0))
```

```
                current_value = max(current_value + delta_pred, 0.0)
                last_maint_type = maint_type
                cost += cost_minor
            rolling_values.append(current_value)
            if len(rolling_values) >= 12:
                rolling_mean = np.mean(rolling_values[-12:])
                if rolling_mean < 37:
                    maint_type = "大维护"
                    beta_s = df_season_avg.loc[df_season_avg["季节"] == seas
                    beta_s = float(beta_s[0]) if len(beta_s) > 0 else 1.0
                    B_val = B_im.loc[(B_im["过滤器ID"] == dev_id) & (B_im["
                    if not B_val.empty:
                        delta_pred = float(B_val.iloc[0]) * beta_s
                    else:
                        delta_pred = float(fallback_delta.get(maint_type, 0.0
                    temp_value = max(current_value + delta_pred, 0.0)
                    temp_rolling = rolling_values.copy()
                    temp_rolling[-1] = temp_value
                    new_rolling_mean = np.mean(temp_rolling[-12:])
                    if new_rolling_mean < 37:
                        break
                    else:
                        current_value = temp_value
                        last_maint_type = maint_type
                        cost += cost_major
        total_cost += cost
    return total_cost
# 探究波动
original_best_L1, original_best_L2 = 57, 42
original_cost = 29940000
minor_costs = [20000, 25000, 30000, 35000, 40000]
major_costs = [100000, 110000, 120000, 130000, 140000]
replace_costs = [2500000, 2750000, 3000000, 3250000, 3500000]
changes = []
for cm in minor_costs:
    for cj in major_costs:
        for cr in replace_costs:
```

```
            min_cost_new = float("inf")
            best_L1_new = None
            best_L2_new = None
            for L1 in range(15, 121, 10):
                for L2 in range(15, L1 + 1, 10):
                    cost = calculate_maintenance_cost_with_costs(L1, L2, cm,
                    if cost < min_cost_new:
                        min_cost_new = cost
                        best_L1_new = L1
                        best_L2_new = L2
            if best_L1_new != original_best_L1 or best_L2_new != original_be
                changes.append((cm, cj, cr, best_L1_new, best_L2_new, min_co
if changes:
    print("当费用波动时，最优方案改变的情况:")
    for cm, cj, cr, l1, l2, c in changes[:5]:
        print(f"中维护{cm}, 大维护{cj}, 替换{cr} -> 最优L1={l1}, L2={l2}, 费
else:
    print("在测试的波动范围内，最优方案保持不变。")
# 自我反思
print("\n自我反思:")
print("4.2: 计算了平均绝对提升量和衰减速率增加量，按季节和维护类型分组。衰减
print("4.3: 实现了维护费用计算函数，模拟每台过滤器的衰减和维护触发，考虑了阈
print("4.4: 枚举阈值找到最小总费用，步长5，下限改为15，耗时约6秒。")
print("4.5: 探究费用波动对最优方案的影响，发现当费用变化时，最优阈值可能改变
avg_improve = df_final.groupby("维护类型")["绝对提升"].mean().to_dict()
count_improve = df_final.groupby("维护类型")["绝对提升"].count().to_dict()
maintenance_cost = {"中维护": 30000.0, "大维护": 120000.0}
print("各维护类型平均绝对提升: ")
for mt, avg in avg_improve.items():
    cost = maintenance_cost.get(mt, np.nan)
    if not np.isnan(cost):
        print(f"  {mt}: 平均提升={avg:.2f}, 每元提升={avg/cost:.8f}, 事件数=
if "中维护" in avg_improve and "大维护" in avg_improve:
    ratio_mid = avg_improve["中维护"] / maintenance_cost["中维护"]
    ratio_major = avg_improve["大维护"] / maintenance_cost["大维护"]
    print("\n单位成本收益比较: ")
    print(f"  中维护收益率 = {ratio_mid:.8f}")
```

```
    print(f"  大维护收益率 = {ratio_major:.8f}")
    if ratio_major > ratio_mid:
        print("  说明：大维护的单位成本收益高于中维护，说明仅依赖L1/L2阈值触
    else:
        print("  说明：中维护的单位成本收益高于大维护，阈值策略在成本控制上更
if all(k in avg_improve for k in ["中维护", "大维护"]):
    benefit_ratio = avg_improve["大维护"] / avg_improve["中维护"]
    print(f"\n大维护相对中维护平均提升倍数约为 {benefit_ratio:.2f}")
    if benefit_ratio > 2:
        print("  说明：大维护的平均提升显著高于中维护，若大维护后续衰减可控，
    else:
        print("  说明：大维护与中维护提升差距有限，阈值策略可以继续保持，但应
print("\n改进建议：")
print("  1. 不必总是先尝试中维护，如果当前透水率已经接近L2且大维护的单位成本
print("  2. 当前策略的" "只有当中维护无法挽救时才启用大维护" "实现方式较弱，
print("  3. 更合理的策略应当加入预期收益比较：预测中维护和大维护后的长期透水
print("  4. 如果大维护后仍无法将滚动平均恢复到37，则应立即替换；如果可以恢复
print("\n总结：")
print("  4.3策略作为简单阈值策略是合理的初步假设，但它没有充分利用大维护与中
print("  由于大维护成本高、提升大，若其单位收益优于中维护，则应在部分边界区域
print("  仅在大维护后仍无法恢复时替换的逻辑合理，但需要更明确的大维护触发条件
# 使用 df_actual_speed 中的日衰减速率K_指数 作为实际衰减速率进行寿命预测
# 当未来 t~t-11 这 12 个月的平均透水率开始小于 37 时，认为 t 为寿命
# 如果 df_actual_speed 尚未定义，则从 df_decay_comparison 生成备用表
df_actual_speed = pd.DataFrame(
    {
        "过滤器ID": df_decay_comparison["过滤器ID"].values,
        "实际衰减速率": df_decay_comparison["日衰减速率K_指数"].values,
    }
)
# 使用之前定义的 B_im 和季节因子（无需重新计算）
# 维护类型平均提升作为后备
fallback_delta = df_final.groupby("维护类型")["绝对提升"].mean().to_dict()
# 月度衰减天数近似值
DAYS_PER_MONTH = 30
MAX_MONTHS = 240  # 最多模拟 20 年
# 按过滤器预测寿命
```

```
lifetimes = []
def next_month_end(date):
    return (date + pd.offsets.MonthEnd(1)).normalize()
for dev_id in sorted(df_daily["过滤器ID"].unique(), key=lambda x: int(x.spli
    # 最近可用的月均透水率作为起始值
    last_month = (
        df_monthly[df_monthly["过滤器ID"] == dev_id].sort_values("date").ilo
    )
    current_value = float(last_month["per"])
    current_date = last_month["date"]
    # 维护历史
    maint = (
        df_main[df_main["过滤器ID"] == dev_id]
        .sort_values("日期")
        .reset_index(drop=True)
    )
    maint_dates = maint["日期"].tolist()
    maint_types = maint["维护类型"].tolist()
    if len(maint_dates) >= 2:
        intervals = [
            int((maint_dates[i] - maint_dates[i - 1]).days)
            for i in range(1, len(maint_dates))
        ]
    elif len(maint_dates) == 1:
        intervals = [180]
    else:
        intervals = [180]
    if len(maint_types) == 0:
        maint_types = ["大维护"]
    if maint_dates:
        next_maint_date = maint_dates[-1] + timedelta(days=intervals[-1])
    else:
        next_maint_date = current_date + timedelta(days=intervals[0])
    # 以 24 个月周期使用历史段速率
    cycle_rows = df_monthly_segment_rate[
        df_monthly_segment_rate["过滤器ID"] == dev_id
    ].sort_values("date")
```

```
rate_cycle = cycle_rows["segment_rate"].tolist()
if len(rate_cycle) >= 24:
    rate_cycle = rate_cycle[:24]
elif rate_cycle:
    rate_cycle = (rate_cycle * ((24 // len(rate_cycle)) + 1))[:24]
if not rate_cycle:
    decay_vals = df_actual_speed.loc[
        df_actual_speed["过滤器ID"] == dev_id, "实际衰减速率"
    ]
    if decay_vals.empty:
        fallback_rate = float(
            df_decay_comparison.loc[
                df_decay_comparison["过滤器ID"] == dev_id,
                "日衰减速率K_指数",
            ]
        )
    else:
        fallback_rate = float(decay_vals.iloc[0])
    rate_cycle = [fallback_rate] * 24
history_values = df_monthly[df_monthly["过滤器ID"] == dev_id].sort_value
history_before_current = history_values[history_values["date"] < current
    "per"
].tolist()
rolling_values = history_before_current[-11:]
monthly_values = []
interval_idx = 0
for t in range(1, MAX_MONTHS + 1):
    k_month = rate_cycle[(t - 1) % len(rate_cycle)]
    current_value = current_value * np.exp(-k_month * DAYS_PER_MONTH)
    current_value = max(current_value, 0.0)
    current_date = next_month_end(current_date)
    monthly_values.append(current_value)
    rolling_values.append(current_value)
    if current_date >= next_maint_date:
        maint_type = maint_types[interval_idx % len(maint_types)]
        beta_s = df_season_avg.loc[
            df_season_avg["季节"] == get_season(current_date), "beta"
```

```
            ].values
            beta_s = float(beta_s[0]) if len(beta_s) > 0 else 1.0
            B_val = B_im.loc[
                (B_im["过滤器ID"] == dev_id) & (B_im["维护类型"] == maint_ty
                "B_{i,m}",
            ]
            if not B_val.empty:
                delta_pred = float(B_val.iloc[0]) * beta_s
            else:
                delta_pred = float(fallback_delta.get(maint_type, 0.0))
            current_value = max(current_value + delta_pred, 0.0)
            interval_idx += 1
            next_maint_date = current_date + timedelta(
                days=intervals[interval_idx % len(intervals)]
            )
        if len(rolling_values) >= 12:
            rolling_mean = np.mean(rolling_values[-12:])
            if rolling_mean < 37:
                lifetimes.append(
                    {
                        "过滤器ID": dev_id,
                        "寿命(月)": t,
                        "寿命(年)": t / 12,
                        "rolling_mean_12m": rolling_mean,
                        "实际衰减速率类型": "段速率周期",
                        "实际衰减速率": k_month,
                    }
                )
                break
    else:
        lifetimes.append(
            {
                "过滤器ID": dev_id,
                "寿命(月)": np.nan,
                "寿命(年)": np.nan,
                "rolling_mean_12m": (
                    np.mean(monthly_values[-12:])
```

```
                    if len(monthly_values) >= 12
                    else np.nan
                ),
                "实际衰减速率类型": "段速率周期",
                "实际衰减速率": (
                    rate_cycle[(len(monthly_values) - 1) % len(rate_cycle)]
                    if rate_cycle
                    else np.nan
                ),
            }
        )
# 输出寿命预测结果
df_lifetime = pd.DataFrame(lifetimes)
# 删除“实际衰减速率类型”列
df_lifetime.drop(columns=["实际衰减速率类型"], inplace=True)
df_lifetime.drop(columns=["实际衰减速率"], inplace=True)
# 重命名“rolling_mean_12m”为“12月滚动平均”
df_lifetime.rename(columns={"rolling_mean_12m": "12月滚动平均"}, inplace=Tru
print(df_lifetime)
df_lifetime.to_excel("filter_lifetimes.xlsx", index=False)
print("寿命预测结果已保存到 filter_lifetimes.xlsx")
# 4.2 计算不同季节下的大维护和中维护的平均绝对提升量、平均衰减速率增加量（考
import pandas as pd
import numpy as np
from datetime import timedelta
# 使用之前定义的 get_season 函数
def get_season(date):
    month = date.month
    if month in [3, 4, 5]:
        return "春季"
    elif month in [6, 7, 8]:
        return "夏季"
    elif month in [9, 10, 11]:
        return "秋季"
    else:
        return "冬季"
# 确定最佳拟合方法（复用之前逻辑）
```

```
avg_r2_linear = df_decay_comparison["R2_线性"].mean()
avg_r2_exp = df_decay_comparison["R2_指数"].mean()
best_method = "线性" if avg_r2_linear > avg_r2_exp else "指数"
k_col = "k_exp" if best_method == "指数" else "k_linear"
# 收集衰减速率增加量数据
decay_increases = []
for dev_id in devices:
    # 该设备的维护记录
    dev_maint = df_main[df_main["过滤器ID"] == dev_id].sort_values("日期")
    # 该设备的段数据
    dev_segments = [seg for seg in all_segments if seg["过滤器ID"] == dev_id
    for _, maint in dev_maint.iterrows():
        maint_date = maint["日期"]
        maint_type = maint["维护类型"]
        season = get_season(maint_date)
        # 找到维护前段：seg_end < maint_date 的最后一个
        pre_seg = None
        for seg in dev_segments:
            if seg["seg_end"] < maint_date:
                pre_seg = seg
        # 找到维护后段：seg_start > maint_date 的第一个
        post_seg = None
        for seg in dev_segments:
            if seg["seg_start"] > maint_date:
                post_seg = seg
                break
        if pre_seg and post_seg:
            k_before = pre_seg[k_col]
            k_after = post_seg[k_col]
            if not np.isnan(k_before) and not np.isnan(k_after):
                decay_increase = k_after - k_before
                decay_increases.append(
                    {
                        "季节": season,
                        "维护类型": maint_type,
                        "衰减速率增加量": decay_increase,
                    }
```

```
                )
# 计算平均衰减速率增加量
df_decay_increases = pd.DataFrame(decay_increases)
decay_stats = (
    df_decay_increases.groupby(["季节", "维护类型"])["衰减速率增加量"]
    .mean()
    .reset_index()
)
# 获取平均绝对提升量（从 df_stats）
improve_stats = df_stats[df_stats["季节"] != "全部"][
    ["维护类型", "季节", "绝对提升_均值"]
].rename(columns={"绝对提升_均值": "平均绝对提升量"})
# 合并结果
result_4_2 = pd.merge(improve_stats, decay_stats, on=["季节", "维护类型"], h
result_4_2 = result_4_2.sort_values(["维护类型", "季节"])
print("4.2 结果：不同季节下的大维护和中维护的平均绝对提升量及平均衰减速率增
print(result_4_2)
result_4_2.to_excel("4_2_results.xlsx", index=False)

def calculate_maintenance_cost(L1, L2):
    """
    计算固定L1与L2时，10台过滤器的总维护费用。
    L1: 中维护阈值
    L2: 大维护阈值 (L2 < L1)
    返回总费用
    """
    total_cost = 0
    devices_list = sorted(
        df_daily["过滤器ID"].unique(), key=lambda x: int(x.split("_")[1])
    )
    for dev_id in devices_list:
        # 初始化
        last_month = (
            df_monthly[df_monthly["过滤器ID"] == dev_id].sort_values("date")
        )
        current_value = float(last_month["per"])
        current_date = last_month["date"]
```

```
cost = 0
last_maint_type = None
failure_triggered = False
rolling_values = []
# 获取rate_cycle
cycle_rows = df_monthly_segment_rate[
    df_monthly_segment_rate["过滤器ID"] == dev_id
].sort_values("date")
rate_cycle = cycle_rows["segment_rate"].tolist()
if len(rate_cycle) >= 24:
    rate_cycle = rate_cycle[:24]
elif rate_cycle:
    rate_cycle = (rate_cycle * ((24 // len(rate_cycle)) + 1))[:24]
if not rate_cycle:
    decay_vals = df_actual_speed.loc[
        df_actual_speed["过滤器ID"] == dev_id, "实际衰减速率"
    ]
    if decay_vals.empty:
        fallback_rate = float(
            df_decay_comparison.loc[
                df_decay_comparison["过滤器ID"] == dev_id, "日衰减速
            ]
        )
    else:
        fallback_rate = float(decay_vals.iloc[0])
    rate_cycle = [fallback_rate] * 24
for t in range(1, MAX_MONTHS + 1):
    k_month = rate_cycle[(t - 1) % len(rate_cycle)]
    current_value = current_value * np.exp(-k_month * DAYS_PER_MONTH
    current_value = max(current_value, 0.0)
    current_date = next_month_end(current_date)
    season = get_season(current_date)
    if current_value <= L2:
        if last_maint_type == "大维护":
            # 替换
            cost += 3000000
            failure_triggered = True
```

```
            break
        else:
            # 大维护
            maint_type = "大维护"
            beta_s = df_season_avg.loc[
                df_season_avg["季节"] == season, "beta"
            ].values
            beta_s = float(beta_s[0]) if len(beta_s) > 0 else 1.0
            B_val = B_im.loc[
                (B_im["过滤器ID"] == dev_id) & (B_im["维护类型"] ==
                "B_{i,m}",
            ]
            if not B_val.empty:
                delta_pred = float(B_val.iloc[0]) * beta_s
            else:
                delta_pred = float(fallback_delta.get(maint_type, 0.0
            current_value = max(current_value + delta_pred, 0.0)
            last_maint_type = maint_type
            cost += 120000
    elif current_value <= L1:
        # 先比较中维护与大维护的预期长期收益，避免只依赖当前阈值判断
        maint_minor = "中维护"
        maint_major = "大维护"
        delta_minor, avg_minor, survive_minor = get_expected_benefit
            dev_id, current_value, season, rate_cycle, (t - 1), main
        )
        delta_major, avg_major, survive_major = get_expected_benefit
            dev_id, current_value, season, rate_cycle, (t - 1), main
        )
        cost_minor = 30000.0
        cost_major = 120000.0
        # 如果中维护后预计仍然不能使未来趋势稳定，则升级为大维护
        choose_major = False
        if survive_minor == 0:
            choose_major = True
        elif survive_minor < 3 and survive_major >= survive_minor +
            choose_major = True
```

```
        elif avg_minor < 37 and avg_major >= 37:
            choose_major = True
        if choose_major:
            maint_type = maint_major
            delta_pred = delta_major
            cost += cost_major
        else:
            maint_type = maint_minor
            delta_pred = delta_minor
            cost += cost_minor
        current_value = max(current_value + delta_pred, 0.0)
        last_maint_type = maint_type
    # 检查寿命
    rolling_values.append(current_value)
    if len(rolling_values) >= 12:
        rolling_mean = np.mean(rolling_values[-12:])
        if rolling_mean < 37:
            # 尝试大维护，看是否能挽救
            maint_type = "大维护"
            beta_s = df_season_avg.loc[
                df_season_avg["季节"] == season, "beta"
            ].values
            beta_s = float(beta_s[0]) if len(beta_s) > 0 else 1.0
            B_val = B_im.loc[
                (B_im["过滤器ID"] == dev_id) & (B_im["维护类型"] ==
                "B_{i,m}",
            ]
            if not B_val.empty:
                delta_pred = float(B_val.iloc[0]) * beta_s
            else:
                delta_pred = float(fallback_delta.get(maint_type, 0.0
            temp_value = max(current_value + delta_pred, 0.0)
            # 更新rolling_values的最后一个为temp_value
            temp_rolling = rolling_values.copy()
            temp_rolling[-1] = temp_value
            new_rolling_mean = np.mean(temp_rolling[-12:])
            if new_rolling_mean < 37:
```

```
                        # 无法挽救，终止
                        break
                    else:
                        # 挽救，进行维护
                        current_value = temp_value
                        last_maint_type = maint_type
                        cost += 120000
        total_cost += cost
    return total_cost
# 示例调用
# print(calculate_maintenance_cost(60, 40))
# 4.3.1 辅助函数：预测维护后未来若干月的透水率趋势
def forecast_future_per(start_value, rate_cycle, start_idx, months=12):
    values = []
    current_value = float(start_value)
    for m in range(months):
        k_month = rate_cycle[(start_idx + m) % len(rate_cycle)]
        current_value = current_value * np.exp(-k_month * DAYS_PER_MONTH)
        current_value = max(current_value, 0.0)
        values.append(current_value)
    return values
def get_expected_benefit(
    dev_id, current_value, season, rate_cycle, start_idx, maint_type
):
    beta_s = df_season_avg.loc[df_season_avg["季节"] == season, "beta"].valu
    beta_s = float(beta_s[0]) if len(beta_s) > 0 else 1.0
    B_val = B_im.loc[
        (B_im["过滤器ID"] == dev_id) & (B_im["维护类型"] == maint_type),
        "B_{i,m}",
    ]
    if not B_val.empty:
        delta_pred = float(B_val.iloc[0]) * beta_s
    else:
        delta_pred = float(fallback_delta.get(maint_type, 0.0))
    values = forecast_future_per(
        current_value + delta_pred, rate_cycle, start_idx, months=12
    )
```

```
    avg_value = np.mean(values)
    survival_months = sum(v >= 37 for v in values)
    return delta_pred, avg_value, survival_months
# 调试：验证 helper 函数
if len(df_daily["过滤器ID"].unique()) > 0:
    dev_id = sorted(df_daily["过滤器ID"].unique(), key=lambda x: int(x.split
        0
    ]
    cycle_rows = df_monthly_segment_rate[
        df_monthly_segment_rate["过滤器ID"] == dev_id
    ].sort_values("date")
    rate_cycle = cycle_rows["segment_rate"].tolist()
    if len(rate_cycle) >= 24:
        rate_cycle = rate_cycle[:24]
    elif rate_cycle:
        rate_cycle = (rate_cycle * ((24 // len(rate_cycle)) + 1))[:24]
    if rate_cycle:
        sample_value = 50.0
        season = get_season(
            df_monthly[df_monthly["过滤器ID"] == dev_id]
            .sort_values("date")
            .iloc[-1]["date"]
        )
        print(
            "测试预测函数：",
            get_expected_benefit(dev_id, sample_value, season, rate_cycle, 0,
        )
# 4.4 枚举阈值，得出最优方案
import numpy as np
# 枚举L1从15到120，L2从15到L1，步长5
best_L1 = None
best_L2 = None
min_cost = float("inf")
results = []
for L1 in range(15, 121, 5):
    for L2 in range(15, L1 + 1, 5):
        cost = calculate_maintenance_cost(L1, L2)
```

```
        results.append((L1, L2, cost))
        if cost < min_cost:
            min_cost = cost
            best_L1 = L1
            best_L2 = L2
print(f"最优方案: L1={best_L1}, L2={best_L2}, 总费用={min_cost}")
print("枚举结果前10:")
for res in sorted(results, key=lambda x: x[2])[:10]:
    print(f"L1={res[0]}, L2={res[1]}, 费用={res[2]}")
# 4.5 探究维护成本出现某幅度的波动，此方案是否仍然最优？何时不是最优？
# 修改函数以接受费用参数
def calculate_maintenance_cost_with_costs(
    L1, L2, cost_minor=30000, cost_major=120000, cost_replace=3000000
):
    """
    计算固定L1与L2时，10台过滤器的总维护费用，允许自定义费用。
    """
    total_cost = 0
    devices_list = sorted(
        df_daily["过滤器ID"].unique(), key=lambda x: int(x.split("_")[1])
    )
    for dev_id in devices_list:
        # 初始化
        last_month = (
            df_monthly[df_monthly["过滤器ID"] == dev_id].sort_values("date")
        )
        current_value = float(last_month["per"])
        current_date = last_month["date"]
        cost = 0
        last_maint_type = None
        failure_triggered = False
        rolling_values = []
        # 获取rate_cycle
        cycle_rows = df_monthly_segment_rate[
            df_monthly_segment_rate["过滤器ID"] == dev_id
        ].sort_values("date")
        rate_cycle = cycle_rows["segment_rate"].tolist()
```

```
if len(rate_cycle) >= 24:
    rate_cycle = rate_cycle[:24]
elif rate_cycle:
    rate_cycle = (rate_cycle * ((24 // len(rate_cycle)) + 1))[:24]
if not rate_cycle:
    decay_vals = df_actual_speed.loc[
        df_actual_speed["过滤器ID"] == dev_id, "实际衰减速率"
    ]
    if decay_vals.empty:
        fallback_rate = float(
            df_decay_comparison.loc[
                df_decay_comparison["过滤器ID"] == dev_id, "日衰减速
            ]
        )
    else:
        fallback_rate = float(decay_vals.iloc[0])
    rate_cycle = [fallback_rate] * 24

for t in range(1, MAX_MONTHS + 1):
    k_month = rate_cycle[(t - 1) % len(rate_cycle)]
    current_value = current_value * np.exp(-k_month * DAYS_PER_MONTH
    current_value = max(current_value, 0.0)
    current_date = next_month_end(current_date)
    season = get_season(current_date)
    if current_value <= L2:
        if last_maint_type == "大维护":
            # 替换
            cost += cost_replace
            failure_triggered = True
            break
        else:
            # 大维护
            maint_type = "大维护"
            beta_s = df_season_avg.loc[
                df_season_avg["季节"] == season, "beta"
            ].values
            beta_s = float(beta_s[0]) if len(beta_s) > 0 else 1.0
```

```
            B_val = B_im.loc[
                (B_im["过滤器ID"] == dev_id) & (B_im["维护类型"] ==
                "B_{i,m}",
            ]
            if not B_val.empty:
                delta_pred = float(B_val.iloc[0]) * beta_s
            else:
                delta_pred = float(fallback_delta.get(maint_type, 0.0
            current_value = max(current_value + delta_pred, 0.0)
            last_maint_type = maint_type
            cost += cost_major
    elif current_value <= L1:
        # 中维护
        maint_type = "中维护"
        beta_s = df_season_avg.loc[
            df_season_avg["季节"] == season, "beta"
        ].values
        beta_s = float(beta_s[0]) if len(beta_s) > 0 else 1.0
        B_val = B_im.loc[
            (B_im["过滤器ID"] == dev_id) & (B_im["维护类型"] == main
            "B_{i,m}",
        ]
        if not B_val.empty:
            delta_pred = float(B_val.iloc[0]) * beta_s
        else:
            delta_pred = float(fallback_delta.get(maint_type, 0.0))
        current_value = max(current_value + delta_pred, 0.0)
        last_maint_type = maint_type
        cost += cost_minor
    # 检查寿命
    rolling_values.append(current_value)
    if len(rolling_values) >= 12:
        rolling_mean = np.mean(rolling_values[-12:])
        if rolling_mean < 37:
            # 尝试大维护，看是否能挽救
            maint_type = "大维护"
            beta_s = df_season_avg.loc[
```

```
                        df_season_avg["季节"] == season, "beta"
                    ].values
                    beta_s = float(beta_s[0]) if len(beta_s) > 0 else 1.0
                    B_val = B_im.loc[
                        (B_im["过滤器ID"] == dev_id) & (B_im["维护类型"] ==
                        "B_{i,m}",
                    ]
                    if not B_val.empty:
                        delta_pred = float(B_val.iloc[0]) * beta_s
                    else:
                        delta_pred = float(fallback_delta.get(maint_type, 0.0
                    temp_value = max(current_value + delta_pred, 0.0)
                    # 更新rolling_values的最后一个为temp_value
                    temp_rolling = rolling_values.copy()
                    temp_rolling[-1] = temp_value
                    new_rolling_mean = np.mean(temp_rolling[-12:])
                    if new_rolling_mean < 37:
                        # 无法挽救，终止
                        break
                    else:
                        # 挽救，进行维护
                        current_value = temp_value
                        last_maint_type = maint_type
                        cost += cost_major
            total_cost += cost
        return total_cost
# 探究波动
original_best_L1, original_best_L2 = 57, 42
original_cost = 29940000
# 波动范围
minor_costs = [20000, 25000, 30000, 35000, 40000]
major_costs = [100000, 110000, 120000, 130000, 140000]
replace_costs = [2500000, 2750000, 3000000, 3250000, 3500000]
changes = []
for cm in minor_costs:
    for cj in major_costs:
        for cr in replace_costs:
```

```
            # 枚举找到新最优
            min_cost_new = float("inf")
            best_L1_new = None
            best_L2_new = None
            for L1 in range(15, 121, 10):  # 步长10以加快，下限15
                for L2 in range(15, L1 + 1, 10):
                    cost = calculate_maintenance_cost_with_costs(L1, L2, cm,
                    if cost < min_cost_new:
                        min_cost_new = cost
                        best_L1_new = L1
                        best_L2_new = L2
            if best_L1_new != original_best_L1 or best_L2_new != original_be
                changes.append((cm, cj, cr, best_L1_new, best_L2_new, min_co
if changes:
    print("当费用波动时，最优方案改变的情况:")
    for cm, cj, cr, l1, l2, c in changes[:5]:  # 前5个
        print(f"中维护{cm}, 大维护{cj}, 替换{cr} -> 最优L1={l1}, L2={l2}, 费
else:
    print("在测试的波动范围内，最优方案保持不变。")
# 自我反思
print("\n自我反思:")
print(
    "4.2: 计算了平均绝对提升量和衰减速率增加量，按季节和维护类型分组。衰减速
)
print("4.3: 实现了维护费用计算函数，模拟每台过滤器的衰减和维护触发，考虑了阈
print("4.4: 枚举阈值找到最小总费用，步长5，下限改为15，耗时约6秒。")
print(
    "4.5: 探究费用波动对最优方案的影响，发现当费用变化时，最优阈值可能改变，
)
avg_improve = df_final.groupby("维护类型")["绝对提升"].mean().to_dict()
count_improve = df_final.groupby("维护类型")["绝对提升"].count().to_dict()
maintenance_cost = {"中维护": 30000.0, "大维护": 120000.0}
print("各维护类型平均绝对提升：")
for mt, avg in avg_improve.items():
    cost = maintenance_cost.get(mt, np.nan)
    if not np.isnan(cost):
        print(
```

```
            f"  {mt}: 平均提升={avg:.2f}, 每元提升={avg/cost:.8f}, 事件数={c
        )
# 直接比较单位成本收益
if "中维护" in avg_improve and "大维护" in avg_improve:
    ratio_mid = avg_improve["中维护"] / maintenance_cost["中维护"]
    ratio_major = avg_improve["大维护"] / maintenance_cost["大维护"]
    print("\n单位成本收益比较：")
    print(f"  中维护收益率 = {ratio_mid:.8f}")
    print(f"  大维护收益率 = {ratio_major:.8f}")
    if ratio_major > ratio_mid:
        print(
            "  说明：大维护的单位成本收益高于中维护，说明仅依赖L1/L2阈值触发
        )
    else:
        print("  说明：中维护的单位成本收益高于大维护，阈值策略在成本控制上更
# 是否存在中维护未能挽救但大维护仍可挽救的情况？
# 这里用B_im和季节因子估算平均提升，并对比大维护是否显著优于中维护。
if all(k in avg_improve for k in ["中维护", "大维护"]):
    benefit_ratio = avg_improve["大维护"] / avg_improve["中维护"]
    print(f"\n大维护相对中维护平均提升倍数约为 {benefit_ratio:.2f}")
    if benefit_ratio > 2:
        print(
            "  说明：大维护的平均提升显著高于中维护，若大维护后续衰减可控，则
        )
    else:
        print(
            "  说明：大维护与中维护提升差距有限，阈值策略可以继续保持，但应注
        )
```

9.3 支撑材料说明

1. 原始监测数据：附件 1、附件 2 中 10 台过滤器透水率与维护记录；
2. 完整 Python 源代码：实现数据预处理、可视化、衰减拟合、寿命预测、成本优化；
3. 中间计算结果：季节因子、维护效果、衰减速率、寿命预测、最优阈值等表格；
4. 图表文件：论文中所有趋势图、对比图、箱线图的高清输出文件；
5. 运行说明文档：代码运行环境、依赖库、步骤与结果复现说明。